

后台开发MySQL技术

QQ: 2491016871

课堂主页: <https://fixbug.ke.qq.com/>

- 本课程主要结合后台开发项目用到的MySQL以及校招面试中出现的MySQL重点知识内容进行讲解
- Win10 + CentOS 7 + MySQL 5.7
- 侧重实践+理论
- MySQL源码

MySQL基础

MySQL介绍

- 关系型数据库
- MySQL目前属于Oracle，大家熟悉的关系型数据库还有SQL Server，Oracle、MySQL，MariaDB，DB2
- MySQL分为企业版和社区版，其中社区版是完全免费并且开源的
- MySQL区别于其它关系型数据库很大的一个特点就是支持插件式的存储引擎，支持如InnoDB，MyISAM，Memory等
- 目前google、腾讯、淘宝、百度、新浪、facebook等公司都在使用MySQL作为数据存储层方案
- MySQL设计成C/S模型
- MySQL的服务器模型采用的是I/O复用+可伸缩的线程池，是实现网络服务器的经典模型

MySQL环境搭建

<https://dev.mysql.com/downloads/>

安装mysql-server服务

Win10安装mysql-server

CentOS 7安装mysql-server

1. 下载rpm安装包

```
[admin@localhost Downloads]$ wget http://repo.mysql.com/mysql57-community-release-e17.rpm
--2020-10-03 11:59:55-- http://repo.mysql.com/mysql57-community-release-e17.rpm
Resolving repo.mysql.com (repo.mysql.com)... 23.45.53.26
Connecting to repo.mysql.com (repo.mysql.com)|23.45.53.26|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 25680 (25K) [application/x-redhat-package-manager]
Saving to: 'mysql57-community-release-e17.rpm'
```

```

100%
[=====>]
25,680      --.-K/s    in 0.09s

2020-10-03 11:59:56 (273 KB/s) - 'mysql57-community-release-el7.rpm' saved
[25680/25680]

[admin@localhost Downloads]$ ls
mysql57-community-release-el7.rpm
[admin@localhost Downloads]$ ll /etc/yum.repos.d/mysql*
ls: cannot access /etc/yum.repos.d/mysql*: No such file or directory

```

```

[root@localhost Downloads]# rpm -ivh mysql57-community-release-el7.rpm
Preparing...                               ##### [100%]
Updating / installing...
   1:mysql57-community-release-el7-11 ##### [100%]
[root@localhost Downloads]# ll /etc/yum.repos.d/mysql*
-rw-r--r--. 1 root root 1838 Apr 27 2017 /etc/yum.repos.d/mysql-community.repo
-rw-r--r--. 1 root root 1885 Apr 27 2017 /etc/yum.repos.d/mysql-community-
source.repo

```

```

[root@localhost Downloads]# yum install mysql-server

```

启动mysql-server

超户模式下启动mysqld服务: `service mysqld start`

```

[root@localhost Downloads]# service mysqld start
Redirecting to /bin/systemctl start mysqld.service

[root@localhost Downloads]# netstat -tanp
Active Internet connections (servers and established)

```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	0.0.0.0:111	0.0.0.0:*	LISTEN	836/rpcbind
tcp	0	0	192.168.122.1:53	0.0.0.0:*	LISTEN	1720/dnsmasq
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	1385/sshd
tcp	0	0	127.0.0.1:631	0.0.0.0:*	LISTEN	1388/cupsd
tcp	0	0	127.0.0.1:6010	0.0.0.0:*	LISTEN	3916/sshd: admin@pt
tcp	0	0	127.0.0.1:6011	0.0.0.0:*	LISTEN	59049/sshd: admin@p
tcp	0	0	192.168.131.129:22	192.168.131.1:62125	ESTABLISHED	59044/sshd: admin [
tcp	0	0	192.168.131.129:22	192.168.131.1:60013	ESTABLISHED	3901/sshd: admin [p
tcp6	0	0	:::3306	:::*	LISTEN	59808/mysqld
tcp6	0	0	:::111	:::*	LISTEN	836/rpcbind

```
tcp6      0      0 :::22                :::*                LISTEN
1385/sshd
tcp6      0      0 :::1:631             :::*                LISTEN
1388/cupsd
tcp6      0      0 :::1:6010            :::*                LISTEN
3916/sshd: admin@pt
tcp6      0      0 :::1:6011            :::*                LISTEN
59049/sshd: admin@p
```

登录mysql-server

安装mysql默认生成的密码在log日志文件中:

```
[root@localhost Downloads]# grep "password" /var/log/mysqld.log
2020-10-03 01:46:39 58097 [Note] Shutting down plugin 'sha256_password'
2020-10-03 01:46:39 58097 [Note] Shutting down plugin 'mysql_old_password'
2020-10-03 01:46:39 58097 [Note] Shutting down plugin 'mysql_native_password'
2020-10-03T04:10:10.401393Z 1 [Note] A temporary password is generated for
root@localhost: fCyQpvz2ao*P
2020-10-03T04:10:17.873968Z 2 [Note] Access denied for user 'root'@'localhost'
(using password: NO)
```

此时通过上面的密码可首次登录mysql server:

```
[root@localhost Downloads]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.7.31

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
ERROR 1820 (HY000): You must reset your password using ALTER USER statement
before executing this statement.
```

根据提示必须修改初始密码并重新测试登录:

```
mysql> set global validate_password_policy=0;
Query OK, 0 rows affected (0.00 sec)

mysql> set global validate_password_length=1;
Query OK, 0 rows affected (0.00 sec)

mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.00 sec)

mysql> quit
```

```
Bye
[root@localhost Downloads]# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.7.31 MySQL Community Server (GPL)

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

目录介绍

/var/lib/mysql/ 数据文件

/etc/my.cnf 配置文件

MySQL数据类型

MySQL数据类型定义了数据的大小范围，因此使用时选择合适的类型，不仅会降低表占用的磁盘空间，间接减少了磁盘I/O的次数，提高了表的访问效率，而且索引的效率也和数据的类型息息相关。

数值类型

整数类型	字节	最小值	最大值
TINYINT	1	有符号-128 无符号 0	有符号 127 无符号 255
SMALLINT	2	有符号-32768 无符号 0	有符号 32767 无符号 65535
MEDIUMINT	3	有符号-8388608 无符号 0	有符号 8388607 无符号 1677215
INT、INTEGER	4	有符号-2147483648 无符号 0	有符号 2147483647 无符号 4294967295
BIGINT	8	有符号-9223372036854775808 无符号 0	有符号 9223372036854775807 无符号 18446744073709551615
浮点数类型	字节	最小值	最大值
FLOAT	4	±1.175494351E-38	±3.402823466E+38
DOUBLE	8	±2.2250738585072014E-308	±1.7976931348623157E+308

浮点类型推荐使用decimal类型（保存为字符串格式）。

字符串类型

字符串类型	字节	描述及存储需求
CHAR (M)	M	M 为 0~255 之间的整数
VARCHAR (M)		M 为 0~65535 之间的整数，值的长度+1 个字节
TINYBLOB		允许长度 0~255 字节，值的长度+1 个字节
BLOB		允许长度 0~65535 字节，值的长度+2 个字节
MEDIUMBLOB		允许长度 0~167772150 字节，值的长度+3 个字节
LONGBLOB		允许长度 0~4294967295 字节，值的长度+4 个字节
TINYTEXT		允许长度 0~255 字节，值的长度+2 个字节
TEXT		允许长度 0~65535 字节，值的长度+2 个字节
MEDIUMTEXT		允许长度 0~167772150 字节，值的长度+3 个字节
LONGTEXT		允许长度 0~4294967295 字节，值的长度+4 个字节
VARBINARY (M)		允许长度 0~M 个字节的变长字节字符串，值的长度+1 个字节
BINARY (M)	M	允许长度 0~M 个字节的定长字节字符串

日期和时间类型

日期和时间类型	字节	最小值	最大值
DATE	4	1000-01-01	9999-12-31
DATETIME	8	1000-01-01 00:00:00	9999-12-31 23:59:59
TIMESTAMP	4	19700101080001	2038 年的某个时刻
TIME	3	-838:59:59	838:59:59
YEAR	1	1901	2155

日期类型也是做项目过程中，经常使用的类型信息，尤其是TIMESTAMP和DATETIME两个类型，但是注意TIMESTAMP会自动更新时间，非常适合那些需要记录最新更新时间的场景，而DATETIME需要手动更新。

enum和set

这两个类型，都是限制该字段只能取固定的值，但是枚举字段只能取一个唯一的值，而集合字段可以取任意个值。

MySQL运算符

算数运算符

运算符	作用
+	加法
-	减法
*	乘法
/, DIV	除法，返回商
%, MOD	除法，返回余数

逻辑运算符

运算符	作用
NOT 或!	逻辑非
AND 或&&	逻辑与
OR 或	逻辑或

比较运算符

运算符	作用
=	等于
<>或!=	不等于
<=>	NULL 安全的等于(NULL-safe)
<	小于
<=	小于等于
>	大于
>=	大于等于
BETWEEN	存在与指定范围
IN	存在于指定集合
IS NULL	为 NULL
IS NOT NULL	不为 NULL
LIKE	通配符匹配
REGEXP 或 RLIKE	正则表达式匹配

MySQL常用函数

字符串函数

数值函数

时间和日期函数

NOW(): 返回当前的日期和时间

UNIX_TIMESTAMP(data): 返回日期date的UNIX时间戳

CURRENT_TIMESTAMP、CURRENT_TIMESTAMP(): 当前时间

CURDATE(): 当前日期

CURTIME(): 当前时间

DATE_FORMAT(date, "%y-%m-%d"): 格式化日期时间等

聚合函数

count、sum、avg、max、min

MySQL完整性约束

主键约束

primary key

自增键约束

auto_increment

唯一键约束

unique

非空约束

not null

默认值约束

default

外键约束

foreign key

关系型数据库表设计

一对一

一对多

多对多

关系型数据库范式

应用数据库范式可以带来许多好处，但是最重要的好处归结为三点：

- 1) 减少数据冗余（这是最主要的好处，其他好处都是由此而附带的）
- 2) 消除异常（插入异常，更新异常，删除异常）
- 3) 让数据组织的更加和谐

但是数据库范式绝对不是越高越好，范式越高，意味着表越多，多表联合查询的机率就越大，SQL的效率就变低。

第一范式（1NF）

每一列保持原子特性

列都是基本数据项，不能够再进行分割，否则设计成一对多的实体关系。例如表中的地址字段，可以再细分为省，市，区等不可再分割（即原子特性）的字段，如下：

		noNF
		EmployeeID
		departmentName
		Name
Employeeaddress		AddressID
addressID		job
city		jobDescription
country		skill
street		departmentDescription
		email

上图的表就是把地址字段分成更详细的city, country, street三个字段, 注意, 不符合第一范式不能称作关系型数据库。

第二范式 (2NF)

属性完全依赖于主键-主要针对联合主键

非主属性完全依赖于主关键字, 如果不是完全依赖主键, 应该拆分成新的实体, 设计成一对多的实体关系。

例如: 选课关系表为SelectCourse(学号, 姓名, 年龄, 课程名称, 成绩, 学分), (学号, 课程名称)是联合主键, 但是学分字段只和课程名称有关, 和学号无关, 相当于只依赖联合主键的其中一个字段, 不符合第二范式。

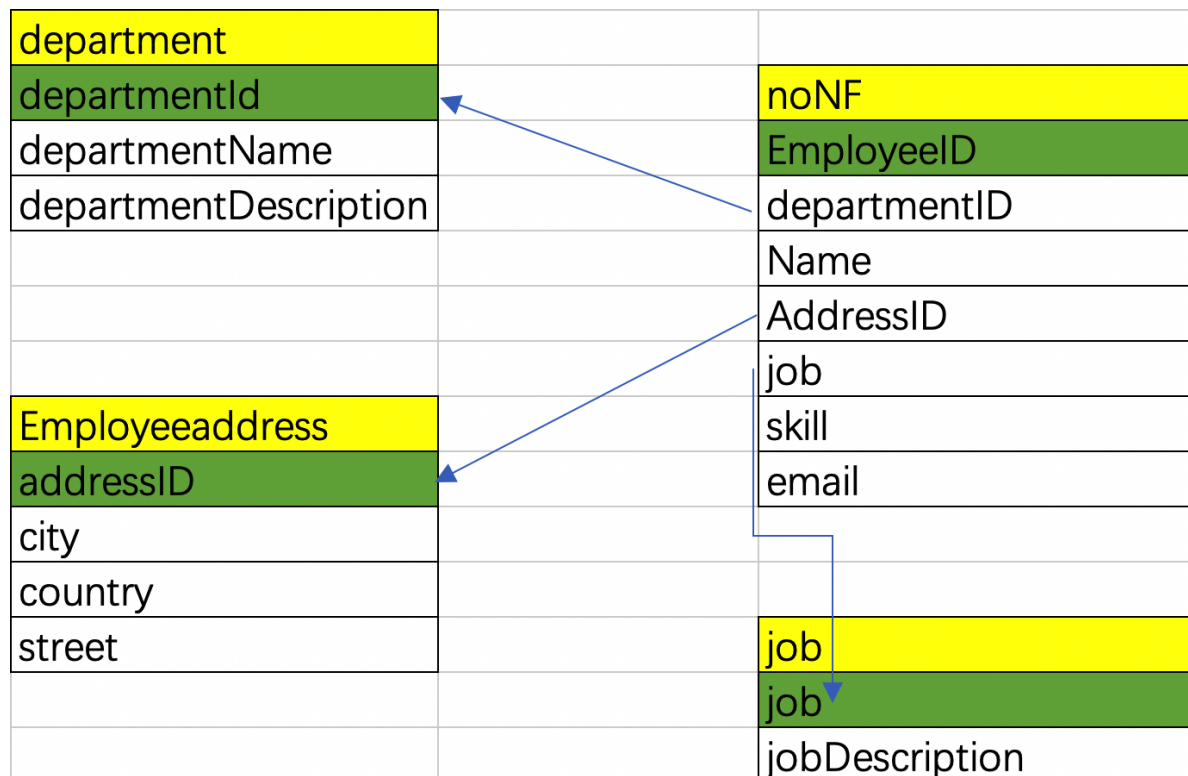
department		
departmentId		noNF
departmentName		EmployeeID
departmentDescription		departmentID
		Name
		AddressID
		job
Employeeaddress		jobDescription
addressID		skill
city		email
country		
street		

第三范式 (3NF)

属性不依赖于其它非主属性

要求一个数据库表中不包含已在其它表中已包含的非主关键字信息。

示例：学生关系表为Student（学号，姓名，年龄，所在学院，学院地点，学院电话），学号是主键，但是学院电话只依赖于所在学院，并不依赖于主键学号，因此该设计不符合第三范式，应该把学院专门设计成一张表，学生表和学院表，两个是一对多的关系。

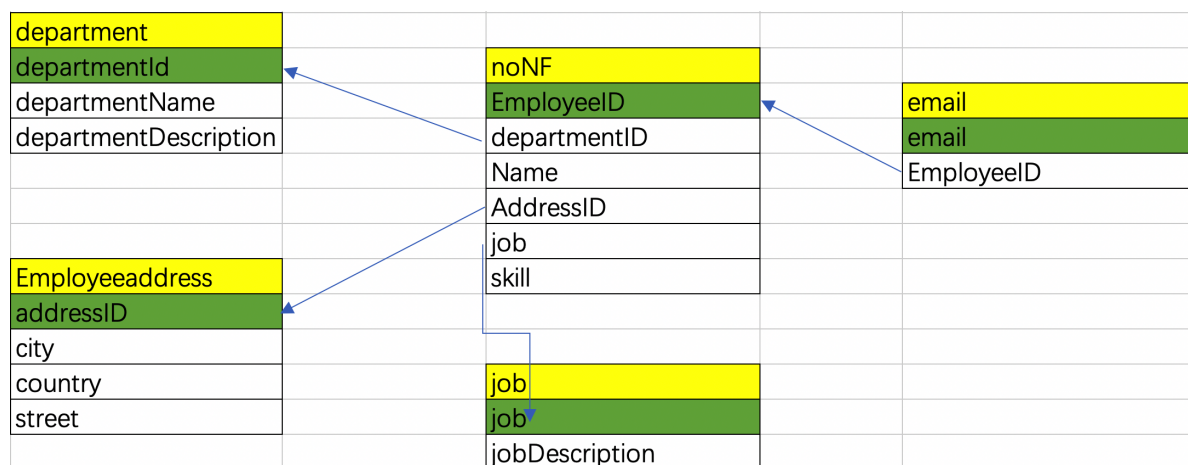


注意：一般关系型数据库满足第三范式就可以了。

BC范式 (BCNF)

每个表中只有一个候选键

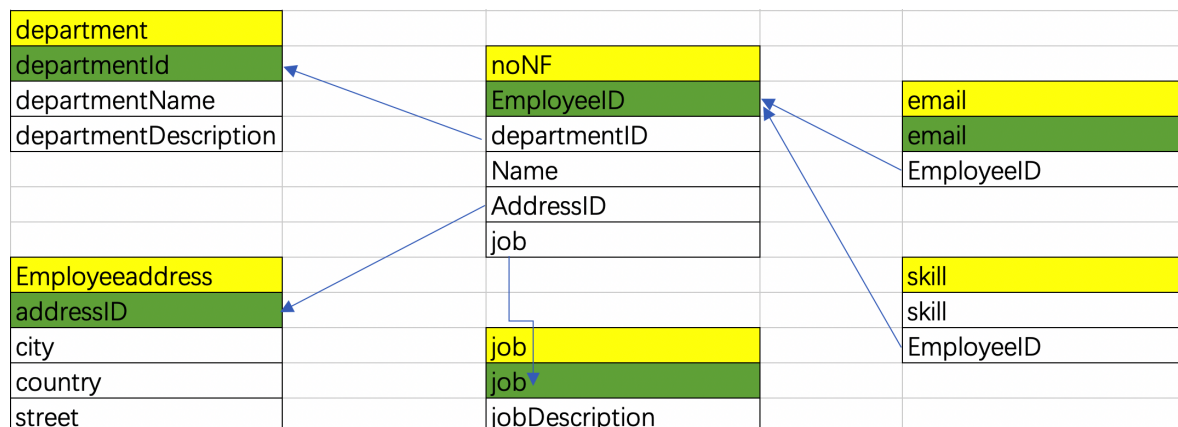
简单的说，BC范式是在第三范式的基础上的一种特殊情况，即每个表中只有一个**候选键**（在一个数据库中每行的值都不相同，则可称为候选键），在上面第三范式的noNF表（上面图3）中可以看出，每一个员工的email都是唯一的（不可能两个人用同一个email），则此表不符合BC范式，对其进行BC范式化后的关系图为：



第四范式 (4NF)

消除表中的多值依赖

简单来说，第四范式就是要消除表中的多值依赖，也就是说可以减少维护数据一致性的工作。比如图4中的noNF表中的skill技能这个字段，有的人是“java, mysql”，有的人描述的是“java, MySQL”，这样数据就不一致了，解决办法就是将多值属性放入一个新表，所以满足第四范式的关系图如下：



从上面对于数据库范式进行分解的过程中不难看出，应用的范式越高，表越多。表多会带来很多问题：

1、查询时需要连接多个表，增加了SQL查询的复杂度

2、查询时需要连接多个表，降低了数据库查询性能

因此，并不是应用的范式越高越好，视实际情况而定。第三范式已经很大程度上减少了数据冗余，并且基本预防了数据插入异常，更新异常，和删除异常了。

MySQL核心SQL

结构化查询语句SQL

SQL是结构化查询语言（Structure Query Language），它是关系型数据库的通用语言。

SQL主要可以划分为以下 3 个类别：

- DDL（Data Definition Languages）语句

数据定义语言，这些语句定义了不同的数据库、表、列、索引等数据库对象的定义。常用的语句关键字主要包括 **create**、**drop**、**alter**等。

- DML（Data Manipulation Language）语句

数据操纵语句，用于添加、删除、更新和查询数据库记录，并检查数据完整性，常用的语句关键字主要包括 **insert**、**delete**、**update** 和 **select** 等。

- DCL（Data Control Language）语句

数据控制语句，用于控制不同的许可和访问级别的语句。这些语句定义了数据库、表、字段、用户的访问权限和安全级别。主要的语句关键字包括 **grant**、**revoke** 等。

库操作

查询数据库

```
show databases;
```

创建数据库

```
create database ChatDB;
```

删除数据库

```
drop database ChatDB;
```

选择数据库

```
use ChatDB;
```

表操作

查看表

```
show tables;
```

创建表

```
create table user(id int unsigned primary key not null auto_increment,  
                  name varchar(50) not null,  
                  age tinyint not null,  
                  sex enum('M','W') not null)engine=INNODB default charset=utf8;
```

查看表结构

```
desc user;
```

查看建表sql

```
show create table user\G
```

删除表

```
drop table user;
```

CRUD操作

insert增加

```
insert into user(nickname, name, age, sex) values('fixbug', 'zhang san', 22,  
'M');  
insert into user(nickname, name, age, sex) values('666', 'li si', 21, 'W'),  
( '888', 'gao yang', 20, 'M');
```

update修改

```
update user set age=23 where name='zhang san';  
update user set age=age+1 where id=3;
```

delete删除

```
delete from user where age=23;
delete from user where age between 20 and 22;
delete from user;
```

select查询

```
select * from user;
select id,nickname,name,age,sex from user;
select id,name from user;
select id,nickname,name,age,sex from user where sex='M' and age>=20 and age<=25;
select id,nickname,name,age,sex from user where sex='M' and age between 20 and 25;
select id,nickname,name,age,sex from user where sex='W' or age>=22;
```

去重distinct

```
select distinct name from user;
```

空值查询

is [not] null

```
select * from user where name is null;
```

union合并查询

SELECT expression1, expression2, ... expression_n

FROM tables[WHERE conditions]

UNION [ALL | DISTINCT] # 注意: union默认去重, 不用修饰distinct, all表示显示所有重复值

SELECT expression1, expression2, ... expression_n

FROM tables[WHERE conditions];

```
SELECT country FROM websites UNION ALL SELECT country FROM apps ORDER BY
country;
```

带in子查询

[NOT] IN(元素1, 元素2, ..., 元素3)

```
select * from user where id in(10, 20, 30, 40, 50)
select * from user where id not in(10, 20, 30, 40, 50)
select * from user where id in(select stu_id from grade where average>=60.0)
```

分页查询

```
select id,nickname,name,age,sex from user limit 10;
select id,nickname,name,age,sex from user limit 2000,10;
```

delimiter \$

```
Create Procedure add_t_user (IN n INT)
BEGIN
DECLARE i INT;
SET i=0;

WHILE i<n DO
INSERT INTO t_user VALUES(NULL,CONCAT(i+1,'@fixbug.com'),i+1);
SET i=i+1;
END WHILE;
END$
```

delimiter ;

call add_t_user(2000000);

排序order by

```
select id,nickname,name,age,sex from user where sex='M' and age>=20 and age<=25
order by age asc;
select id,nickname,name,age,sex from user where sex='M' and age>=20 and age<=25
order by age desc;
```

分组group by

```
select sex from user group by sex;
select count(id),sex from user group by sex;
select count(id),age from user group by age having age>20;
```

笔试实践问题

下表bank_bill是某银行代缴话费的主流水表结构：

字段名	描述
serno	流水号
date	交易日期
accno	账号
name	姓名
amount	金额
brno	缴费网点

- 1、统计表中缴费的总笔数和总金额
- 2、给出一个sql，按网点和日期统计每个网点每天的营业额，并按照营业额进行倒序排序

连接查询

内连接查询

SELECT a.属性名1,a.属性名2,...,b.属性名1,b.属性名2... FROM table_name1 a inner join table_name2 b on a.id = b.id where a.属性名 满足某些条件;

预置条件: uid:1 cid:2

```
select score from exam where uid=1 and cid=2;
```

```
select a.uid,a.name,a.age,a.sex from student a where a.uid=1;
```

```
select c.score from exam c where c.uid=1 and c.cid=2;
```

// on a.uid=c.uid 区分大表 和 小表, 按照数据量来区分, 小表永远是整表扫描, 然后去大表搜索

// 从student小表中取出所有的a.uid, 然后拿着这些uid去exam大表中搜索

// 对于inner join内连接, 过滤条件写在where的后面和on连接条件里面, 效果是一样的

```
select a.uid,a.name,a.age,a.sex,c.score from student a
inner join exam c on a.uid=c.uid where c.uid=1 and c.cid=2;
```

```
select b.cid,b.cname,b.credit from course b where b.cid=2;
```

```
select a.uid,a.name,a.age,a.sex,b.cid,b.cname,b.credit,c.score
from exam c
inner join student a on c.uid=a.uid
inner join course b on c.cid=b.cid
where c.uid=1 and c.cid=2;
```

```
select a.uid,a.name,a.age,a.sex,b.cid,b.cname,b.credit,c.score
from exam c
inner join student a on c.uid=a.uid
inner join course b on c.cid=b.cid
where c.cid=2 and c.score>=90.0;
```

```
select b.cid,b.cname,b.credit,count(*)
from exam c
inner join course b on c.cid=b.cid
where c.score>=90.0
group by c.cid
having c.cid=2;
```

```
select b.cid,b.cname,b.credit,count(*) cnt
from exam c
inner join course b on c.cid=b.cid
where c.score>=90.0
group by c.cid
order by cnt;
```

外连接查询

左连接查询

SELECT a.属性名列表, b.属性名列表 FROM table_name1 a LEFT [OUTER] JOIN table_name2 b on a.id = b.id;

```
// 把left这边的表所有的数据显示出来，在右表中不存在相应数据，则显示NULL
select a.* from User a left outer join Orderlist b on a.uid=b.uid where
a.orderid is null;
```

右连接查询

SELECT a.属性名列表, b.属性名列表 FROM table_name1 a **RIGHT [OUTER] JOIN** table_name2 b on a.id = b.id;

```
// 把right这边的表所有的数据显示出来，在左表中不存在相应数据，则显示NULL
select a.* from User a right outer join Orderlist b on a.uid=b.uid where
b.orderid is null;
```

MySQL存储引擎

MyISAM 不支持事务、也不支持外键，索引采用非聚集索引，其优势是访问的速度快，对事务完整性没有要求，以 SELECT、INSERT 为主的应用基本上都可以使用这个存储引擎来创建表。MyISAM的表在磁盘上存储成 3 个文件，其文件名都和表名相同，扩展名分别是：

.frm（存储表定义）
.MYD（MYData，存储数据）
.MYI（MYIndex，存储索引）

InnoDB 存储引擎提供了具有提交、回滚和崩溃恢复能力的事务安全，支持自动增长列，外键等功能，索引采用聚集索引，索引和数据存储在同一个文件，所以InnoDB的表在磁盘上有两个文件，其文件名都和表名相同，扩展名分别是：

.frm（存储表的定义）
.ibd（存储数据和索引）

MEMORY 存储引擎使用存在内存中的内容来创建表。每个MEMORY 表实际只对应一个磁盘文件，格式是.frm（表结构定义）。MEMORY 类型的表访问非常快，因为它的数据是放在内存中的，并且**默认使用** HASH 索引（不适合做范围查询），但是一旦服务关闭，表中的数据就会丢失掉。

各存储引擎区别

在回答“MySQL不同的存储引擎有哪些区别？”这样的问题时，主要从以下几点来回答就可以（问题的核心）：

种类	锁机制	B-树索引	哈希索引	外键	事务	索引缓存	数据缓存
MyISAM	表锁	支持	不支持	不支持	不支持	支持	不支持
InnoDB	行锁	支持	不支持	支持	支持	支持	支持
Memory	表锁	支持	支持	不支持	不支持	支持	支持

锁机制：表示数据库在并发请求访问的时候，多个事务在操作时，并发操作的粒度。

B-树索引和哈希索引：主要是加速SQL的查询速度。

外键：子表的字段依赖父表的主键，设置两张表的依赖关系。

事务：多个SQL语句，保证它们共同执行的原子操作，要么成功，要么失败，不能只成功一部分，失败需要回滚事务。

索引缓存和数据缓存：和MySQL Server的查询缓存相关，在没有对数据和索引做修改之前，重复查询可以不用进行磁盘I/O（数据库的性能提升，目的是为了减少磁盘I/O操作来提升数据库访问效率），读取上一次内存中查询的缓存就可以了。

MySQL索引

当表中的数据量到达几十万甚至上百万的时候，SQL查询所花费的时间会很长，导致业务超时出错，此时就需要用索引来加速SQL查询。

由于索引也是需要存储成索引文件的，因此对索引的使用也会涉及磁盘I/O操作。如果索引创建过多，使用不当，会造成SQL查询时，进行大量无用的磁盘I/O操作，降低了SQL的查询效率，适得其反，因此掌握良好的索引创建原则非常重要！

索引分类

索引是创建在表上的，是对数据库表中一列或者多列的值进行排序的一种结果。**索引的核心是提高查询的速度！**

物理上(聚集索引&非聚集索引)/逻辑上(...)

索引的优点：提高查询效率

索引的缺点：索引并非越多越好，过多的索引会导致CPU使用率居高不下，由于数据的改变，会造成索引文件的改动，过多的磁盘I/O造成CPU负荷太重

- 1、普通索引：没有任何限制条件，可以给任何类型的字段创建普通索引(创建新表&已创建表，数量是无限的，一张表的一次sql查询只能用一个索引 where a=1 and b='M')
- 2、唯一性索引：使用UNIQUE修饰的字段，值不能够重复，主键索引就隶属于唯一性索引
- 3、主键索引：使用Primary Key修饰的字段会自动创建索引(MyISAM, InnoDB)
- 4、单列索引：在一个字段上创建索引
- 5、多列索引：在表的多个字段上创建索引(uid+cid，多列索引必须使用到第一个列，才能用到多列索引，否则索引用不上)
- 6、全文索引：使用FULLTEXT参数可以设置全文索引，只支持CHAR，VARCHAR和TEXT类型的字段上，常用于数据量较大的字符串类型上，可以提高查询速度(线上项目支持专门的搜索功能，给后台服务器增加专门的搜索引擎支持快速高校的搜索 elasticsearch 简称es C++开源的搜索引擎 搜狗的workflow)

索引创建和删除

创建表的时候指定索引字段：

```
CREATE TABLE index1(id INT,
    name VARCHAR(20),
    sex ENUM('male', 'female'),
    INDEX(id,name));
```

在已经创建的表上添加索引：


```
CREATE [UNIQUE] INDEX 索引名 ON 表名 (属性名 (length) [ASC | DESC]);
```

删除索引:

```
DROP INDEX 索引名 ON 表名;
```

- 1.经常作为where条件过滤的字段考虑添加索引
- 2.字符串列创建索引时，尽量规定索引的长度，而不能让索引值的长度key_len过长
- 3.索引字段涉及类型强转、mysql函数调用、表达式计算等，索引就用不上了

索引的执行过程

explain查看执行计划

使用explain查看sql的执行计划，分析索引的执行过程，mysql的user权限表示例如下：

```
mysql> explain select Host,User from user where Host='%'\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: user
   partitions: NULL
         type: ref
possible_keys: PRIMARY
          key: PRIMARY
        key_len: 180
          ref: const
         rows: 1
   filtered: 100.00
      Extra: Using index
1 row in set, 1 warning (0.00 sec)
```

可以看到使用了主键索引，共扫描1行，Using index表示直接从索引树上查询到结果，不需要回表。

```
mysql> explain select Host,User from user where User='root'\G
***** 1. row *****
      id: 1
  select_type: SIMPLE
        table: user
   partitions: NULL
         type: index
possible_keys: NULL
          key: PRIMARY
        key_len: 276
          ref: NULL
         rows: 4
   filtered: 25.00
      Extra: Using where; Using index
1 row in set, 1 warning (0.00 sec)
```

```
mysql> explain select * from user where User='root'\G
***** 1. row *****
      id: 1
```

```
select_type: SIMPLE
      table: user
    partitions: NULL
            type: ALL
possible_keys: NULL
            key: NULL
          key_len: NULL
            ref: NULL
            rows: 4
        filtered: 25.00
          Extra: Using where
1 row in set, 1 warning (0.00 sec)
```

explain结果字段分析

- select_type

simple: 表示不需要union操作或者不包含子查询的简单select语句。有连接查询时，外层的查询为simple且只有一个。

primary: 一个需要union操作或者含有子查询的select，位于最外层的单位查询的select_type即为primary且只有一个。

union: union连接的两个select查询，除了第一个表外，第二个以后的表的select_type都是union。

union result: 包含union的结果集，在union和union all语句中，因为它不需要参与查询，所以id字段为null。

- table

显示查询的表名；

如果不涉及对数据库操作，这里显示null；

如果显示为尖括号就表示这是个临时表，后边的N就是执行计划中的id，表示结果来自于这个查询产生的；

如果是尖括号括起来<union M,N>也是一个临时表，表示这个结果来自于union查询的id为M，N的结果集；

- type

const: 使用唯一索引或者主键，返回记录一定是1行记录的等值where条件时，通常type就是const。

ref: 常见于辅助索引的等值查找，或者多列主键、唯一索引中，使用第一个列之外的列作为等值查找会出现；返回数据不唯一的等值查找也会出现。

range: 索引范围扫描，常见于使用<、>、is null、between、in、like等运算符的查询中。

index: 索引全表扫描，把索引从头到尾扫一遍；常见于使用索引列就可以处理不需要读取数据文件的查询，可以使用索引排序或者分组的查询。

all: 全表扫描数据文件，然后在server层进行过滤返回符合要求的记录。

- ref

如果使用常数等值查询，这里显示const；

如果是连接查询，被驱动表的执行计划这里会显示驱动表的关联字段；

- Extra

- using filesort: 排序时无法用到索引，常见于order by和group by语句中。

- using index: 查询时不需要回表查询，直接通过索引就可以获取查询的数据。

索引的底层实现原理

数据库索引是存储在磁盘上的，当数据量大时，就不能把整个索引全部加载到内存了，只能逐一加载每一个磁盘块（对应索引树的节点），索引树越低，越“矮胖”，磁盘IO次数就少

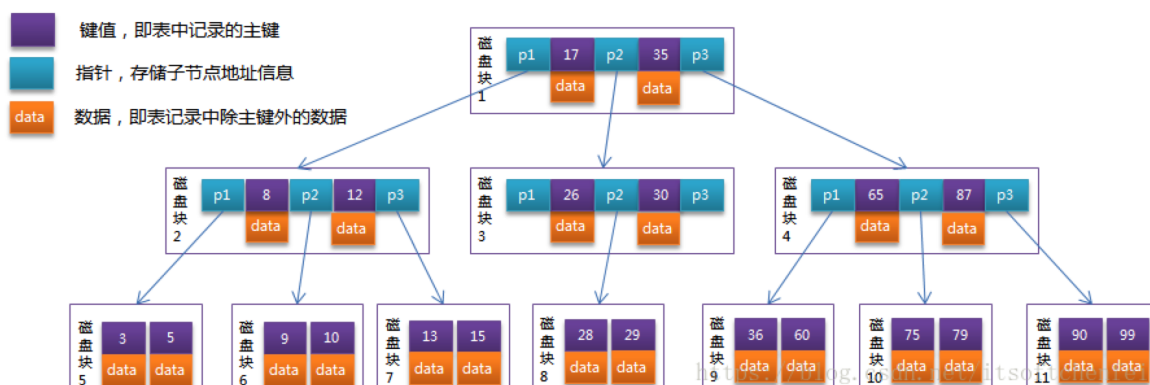
MySQL支持两种索引，一种是B-树索引，一种是哈希索引，大家知道，B-树和哈希表在数据查询时的效率是非常高的。

这里我们主要讨论一下MySQL InnoDB存储引擎，基于B-树（但实际上MySQL采用的是B+树结构）的索引结构。

B-树是一种m阶平衡树，叶子节点都在同一层，由于每一个节点存储的数据量比较大，索引整个B-树的层数是非常低的，基本上不超过三层。

由于磁盘的读取也是按block块操作的（内存是按page页面操作的），因此B-树的节点大小一般设置为和磁盘块大小一致，这样一个B-树节点，就可以通过一次磁盘I/O把一个磁盘块的数据全部存储下来，所以当使用B-树存储索引的时候，磁盘I/O的操作次数是最少的（MySQL的读写效率，主要集中在磁盘I/O上）。

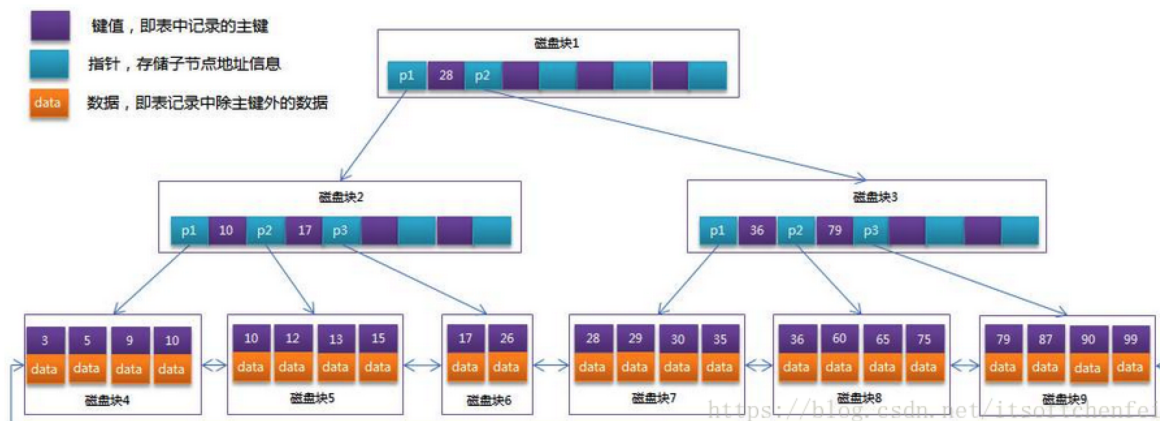
B-树



从上图可以看到B-树存在的缺点：

- 每个节点中有key，也有data，但是每一个节点的存储空间是有限的，如果data数据较大时会导致每个节点能存储的key的数据很小
- 当存储的数据量很大时同样会导致B-树的高度较大，磁盘IO次数花费增大，效率降低

B+树



那么MySQL最终为什么要采用B+树存储索引结构呢，那么看看B-树和B+树在存储结构上有什么不同？

- 1. B-树的每一个节点，存了关键字和对应的数据地址，而B+树的非叶子节点只存关键字，不存数据地址。因此B+树的每一个非叶子节点存储的关键字是远远多于B-树的，B+树的叶子节点存放关键字和数据，因此，从树的高度上来说，B+树的高度要小于B-树，使用的磁盘I/O次数少，因此查询会更快一些。
- 2. B-树由于每个节点都存储关键字和数据，因此离根节点近的数据，查询的就快，离根节点远的数据，查询的就慢；B+树所有的数据都存在叶子节点上，因此在B+树上搜索关键字，找到对应数据的时间是比较平均的，没有快慢之分。
- 3. 在B-树上如果做区间查找，遍历的节点是非常多的；B+树所有叶子节点被连接成了有序链表结构，因此做整表遍历和区间查找是非常容易的。

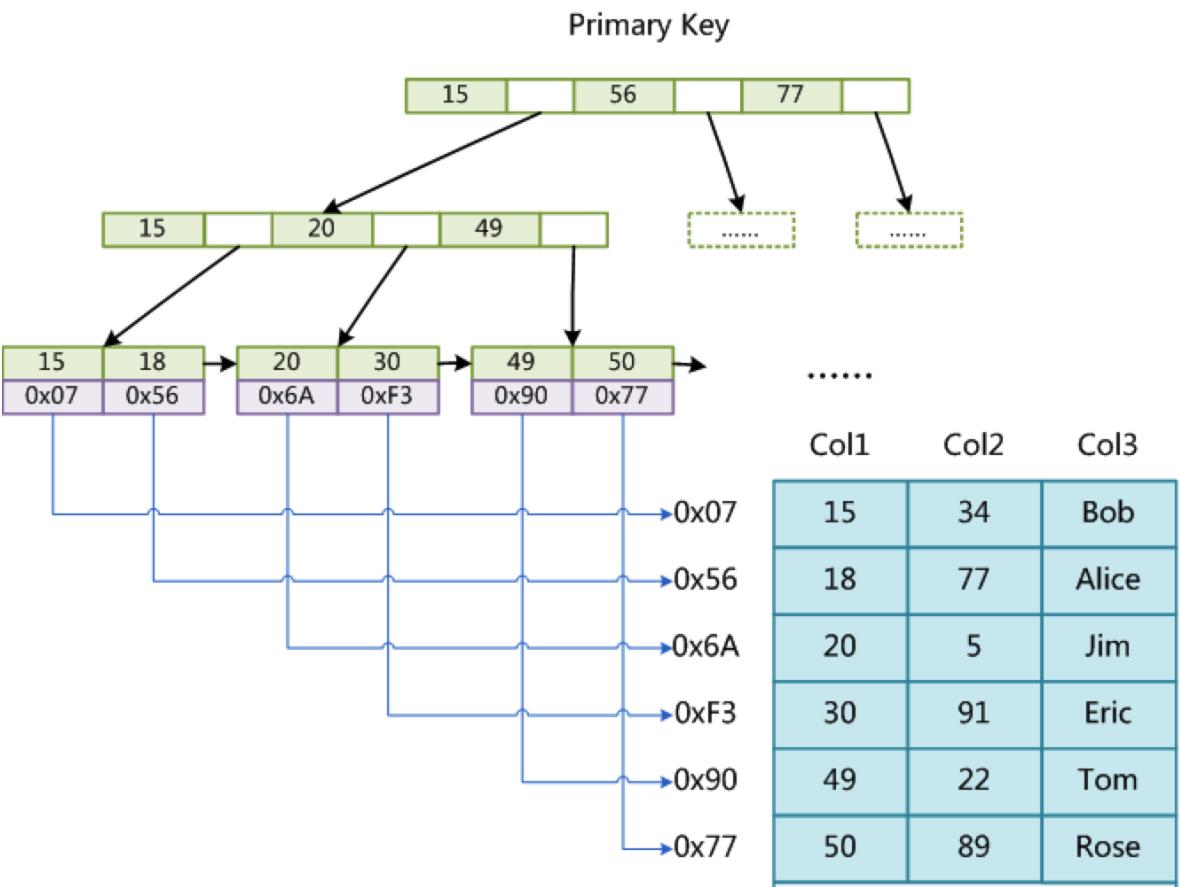
哈希索引当然是由哈希表实现的，哈希表对数据并不排序，因此不适合做区间查找，效率非常低，需要搜索整个哈希表结构。

聚集和非聚集索引

MyISAM

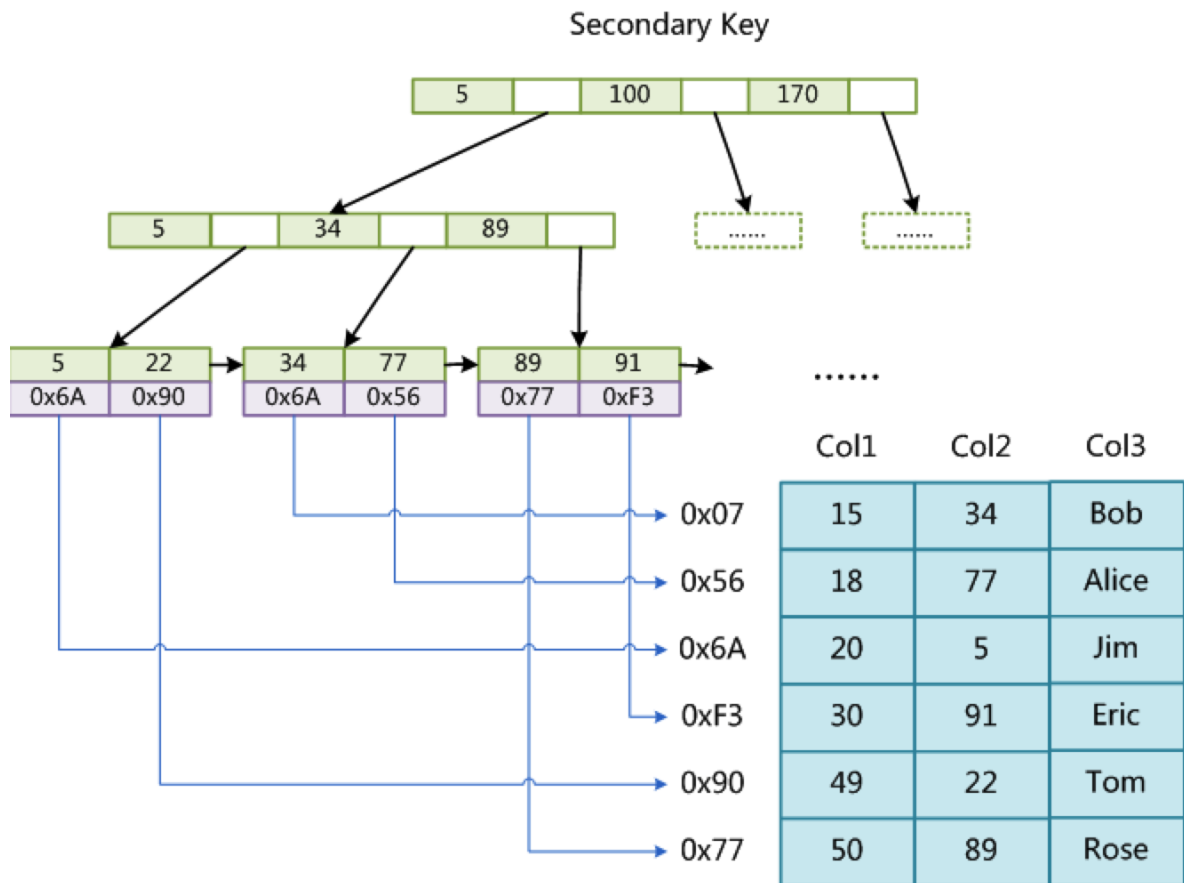
主键索引

MyISAM引擎使用B+树作为索引结构，叶节点的data域存放的是数据记录的地址。下图是MyISAM主键索引的原理图：



辅助索引

在MyISAM中，主索引和辅助索引（Secondary key）在结构上没有任何区别，只是主索引要求key是唯一的，而辅助索引的key可以重复，如果给其它字段创建辅助索引，结构图如下：



根据上面两张图，首先按照B+Tree搜索算法搜索索引，如果指定的Key存在，则取出其data域的值，然后以data域的值作为地址，读取相应数据记录。

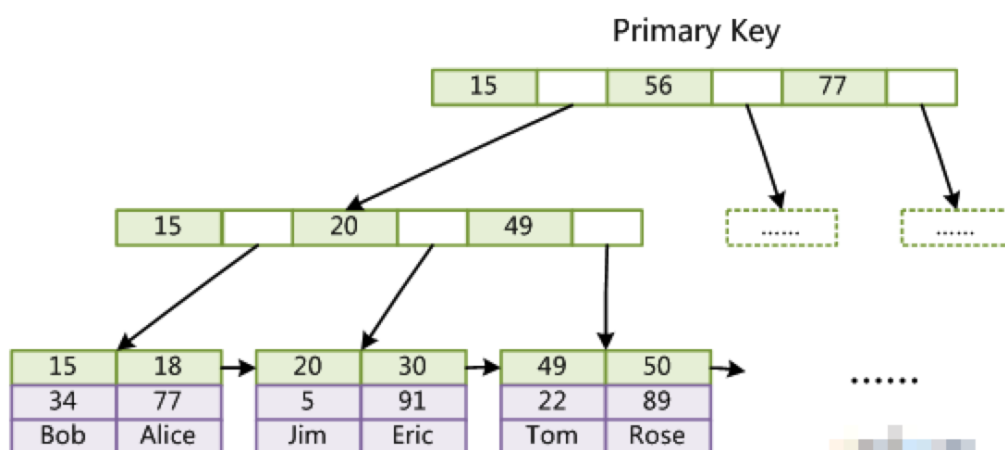
可以看到，MyISAM存储引擎，索引结构叶子节点存储关键字和数据地址，也就是说索引关键字和数据没有在一起存放，体现在磁盘上，就是索引在一个文件存储，数据在另一个文件存储，例如一个user表，会在磁盘上存储三个文件 user.frm（表结构文件） user.MYD（表的数据文件） user.MYI（表的索引文件）。

MyISAM的索引方式也叫做**非聚集索引**。

InnoDB

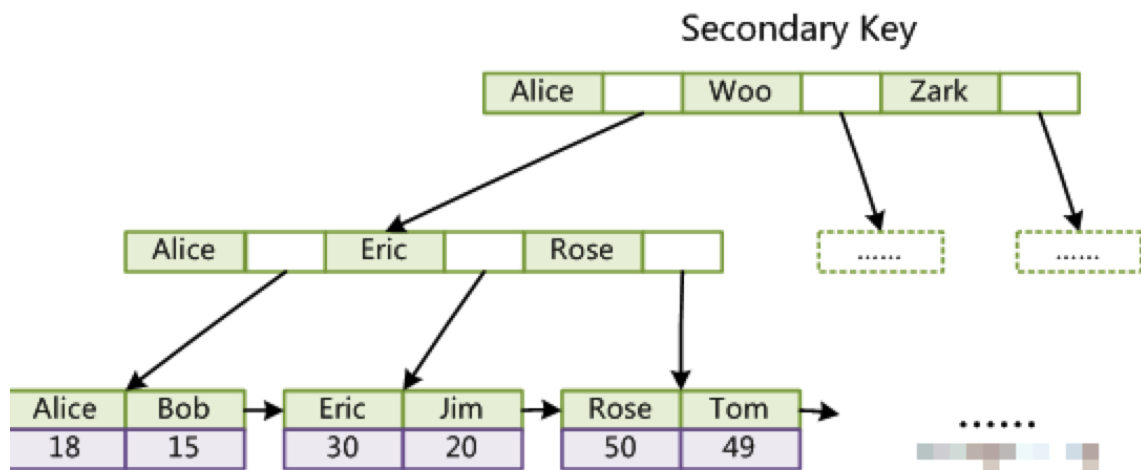
主键索引

InnoDB存储引擎的主键索引，叶子节点中，索引关键字和数据是在一起存放的，如图：



辅助索引

InnoDB的辅助索引，叶子节点上存放的是索引关键字和对应的主键，如图：



辅助索引的B+树，先根据关键字找到对应的主键，再去主键索引树上找到对应的行记录数据。从索引树上可以看到，InnoDB的索引关键字和数据都是在一起存放的，体现在磁盘存储上，例如创建一个user表，在磁盘上只存储两种文件，user.frm（存储表的结构），user.ibd（存储索引和数据）。

InnoDB的索引树叶节点包含了完整的数据记录，这种索引叫做**聚集索引**。因为InnoDB的数据文件本身要按主键聚集，所以InnoDB要求表必须有主键（区别于MyISAM可以没有），如果没有显式指定，则MySQL系统会自动选择一个可以唯一标识数据记录的列作为主键，如果不存在这种列，则MySQL自动为InnoDB表生成一个隐含字段作为主键，这个字段长度为6个字节，类型为长整形。

自适应哈希索引

In MySQL 5.7, the adaptive hash index search system is partitioned. Each index is bound to a specific partition, and each partition is protected by a separate latch.

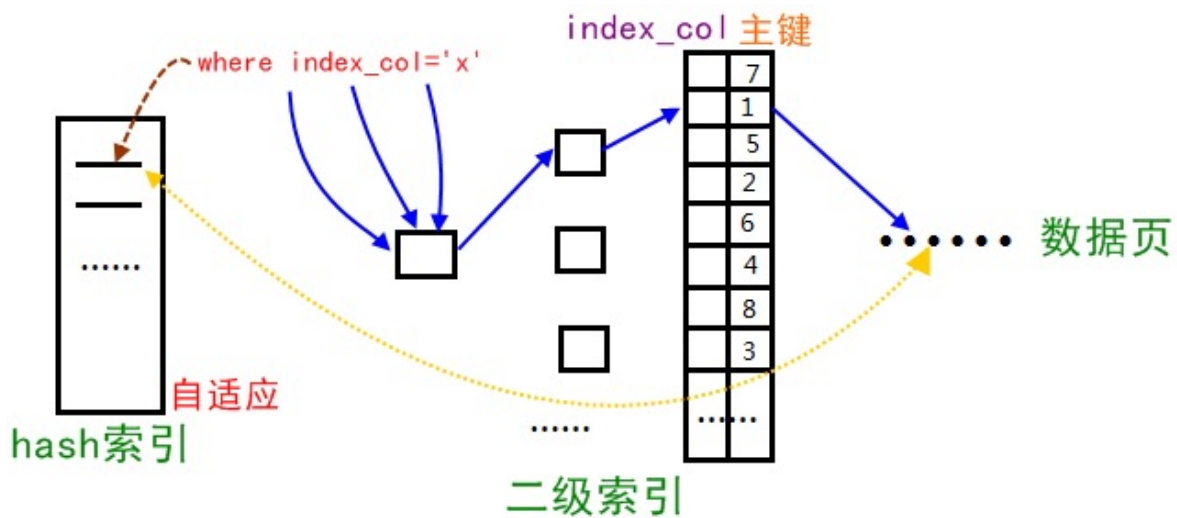
Partitioning is controlled by the `innodb_adaptive_hash_index_parts` configuration option. In earlier releases, the adaptive hash index search system was protected by a single latch which could become a point of contention under heavy workloads. The `innodb_adaptive_hash_index_parts` option is set to 8 by default.

The maximum setting is 512.

The hash index is always built based on an existing B-tree index on the table. InnoDB can build a hash index on a prefix of any length of the key defined for the B-tree,

depending on the pattern of searches that InnoDB observes for the B-tree index. A hash index can be partial, covering only those pages of the index that are often accessed.

You can monitor the use of the adaptive hash index and the contention for its use in the `SEMAPHORES` section of the output of the `SHOW ENGINE INNODB STATUS` command. If you see many threads waiting on an RW-latch created in `btr0sea.c`, then it might be useful to disable adaptive hash indexing.



MySQL事务

事务概念

一个事务是由一条或者多条对数据库操作的SQL语句所组成的一个不可分割的单元，只有当事务中的所有操作都正常执行完了，整个事务才会被提交给数据库；如果有部分事务处理失败，那么事务就要回退到最初的状态，因此，事务要么全部执行成功，要么全部失败。

所以记住事务的几个基本概念，如下：

- 1、事务是一组SQL语句的执行，要么全部成功，要么全部失败，不能出现部分成功，部分失败的结果。保证事务执行的原子操作。
- 2、事务的所有SQL语句全部执行成功，才能提交（commit）事务，把结果写回磁盘上。
- 3、事务执行过程中，有的SQL出现错误，那么事务必须要回滚（rollback）到最初的状态。

ACID特性

每一个事务必须满足下面的4个特性：

事务的原子性（Atomic）：

事务是一个不可分割的整体，事务必须具有原子特性，及当数据修改时，要么全执行，要么全不执行，即不允许事务部分的完成。

事务的一致性（Consistency）：

一个事务执行之前和执行之后，数据库数据必须保持一致性状态。数据库的一致性状态必须由用户来负责，由并发控制机制实现。就拿网上购物来说，你只有让商品出库，又让商品进入顾客的购物车才能构成一个完整的事务。

事务的隔离性（Isolation）：

当两个或者多个事务并发执行时，为了保证数据的安全性，将一个事物内部的操作与其它事务的操作隔离起来，不被其它正在执行的事务所看到，使得并发执行的各个事务之间不能互相影响。

事务的持久性（Durability）：

事务完成(commit)以后，DBMS保证它对数据库中的数据的修改是永久性的，即使数据库因为故障出错，也应该能够恢复数据！

事务并发存在的问题

事务处理不经隔离，并发执行事务时通常会发生以下的问题：

脏读 (Dirty Read)：一个事务读取了另一个事务未提交的数据。例如当事务A和事务B并发执行时，当事务A更新后，事务B查询读取到A尚未提交的数据，此时事务A回滚，则事务B读到的数据就是无效的脏数据。（事务B读取了事务A尚未提交的数据）

不可重复读 (NonRepeatable Read)：一个事务的操作导致另一个事务前后两次读取到不同的数据。例如当事务A和事务B并发执行时，当事务B查询读取数据后，事务A更新操作更改事务B查询到的数据，此时事务B再次去读该数据，发现前后两次读的数据不一样。（事务B读取了事务A已提交的数据）

虚读 (Phantom Read) 幻读：一个事务的操作导致另一个事务前后两次查询的结果数据量不同。例如当事务A和事务B并发执行时，当事务B查询读取数据后，事务A新增或者删除了一条满足事务B查询条件的记录，此时事务B再去查询，发现查询到前一次不存在的记录，或者前一次查询的一些记录不见了。（事务B读取了事务A新增加的数据或者读不到事务A删除的数据）

事务的隔离级别

MySQL支持的四种隔离级别是：

- 1、**TRANSACTION_READ_UNCOMMITTED**。未提交读。说明在提交前一个事务可以看到另一个事务的变化。这样读脏数据，不可重复读和虚读都是被允许的。
- 2、**TRANSACTION_READ_COMMITTED**。已提交读。说明读取未提交的数据是不允许的。这个级别仍然允许不可重复读和虚读产生。
- 3、**TRANSACTION_REPEATABLE_READ**。可重复读。说明事务保证能够再次读取相同的数据而不会失败，但虚读仍然会出现。
- 4、**TRANSACTION_SERIALIZABLE**。串行化。是最高的事务级别，它防止读脏数据，不可重复读和虚读。

隔离级别	脏读	不可重复读	幻读
未提交读	可以	可以	可以
已提交读	不可以	可以	可以
可重复读	不可以	不可以	可以
串行化	不可以	不可以	不可以

备注：

事务隔离级别越高，为避免冲突所花费的性能也就越多。

在“可重复读”级别，实际上可以解决部分的虚读问题，但是不能防止update更新产生的虚读问题，要禁止虚读产生，还是需要设置串行化隔离级别。

MySQL的事务处理命令

打开MySQL的Command命令行窗口，测试以下命令：

- 1、SELECT @@AUTOCOMMIT; 查看MySQL是否自动提交事务

```
mysql> select @@autocommit;
+-----+
| @@autocommit |
+-----+
|             1 |
+-----+
1 row in set (0.00 sec)
```


0表示手动提交事务，1表示自动提交事务，设置事务提交方式为手动提交方式：

```
mysql> set autocommit=0;
Query OK, 0 rows affected (0.00 sec)
```

BEGIN; 开启一个事务

COMMIT; 提交一个事务

ROLLBACK; 回滚一个事务到初始的位置

SAVEPOINT point1; 设置一个名字为point1的保存点

ROLLBACK TO point1; 事务回滚到保存点point1，而不是回滚到初始状态

SET TX_ISOLATION='REPEATABLE-READ'; 设置事务的隔离级别

SELECT @@ TX_ISOLATION; 查询事务的隔离级别

```
mysql> select @@tx_isolation;
+-----+
| @@tx_isolation |
+-----+
| REPEATABLE-READ |      可以看到MySQL默认工作在可重复读级别
+-----+
1 row in set, 1 warning (0.00 sec)
```

MySQL的锁机制

表级锁&行级锁

表级锁：对整张表加锁。开销小，加锁快，不会出现死锁；锁粒度大，发生锁冲突的概率高，并发度低。

行级锁：对某行记录加锁。开销大，加锁慢，会出现死锁；锁定粒度最小，发生锁冲突的概率最低，并发度高。

排它锁和共享锁

排它锁（Exclusive），又称为X锁，写锁。

共享锁（Shared），又称为S锁，读锁。

X和S锁之间有以下的关系：SS可以兼容的，XS、SX、XX之间是互斥的

- 一个事务对数据对象O加了S锁，可以对O进行读取操作但不能进行更新操作。加锁期间其它事务能对O加S锁但不能加X锁。
- 一个事务对数据对象O加了X锁，就可以对O进行读取和更新。加锁期间其它事务不能对O加任何锁。
- 显示加锁：select ... lock in share mode强制获取共享锁，select ... for update获取排它锁

InnoDB行级锁

行级锁

InnoDB存储引擎支持事务处理，表支持行级锁定，并发能力更好。

1、InnoDB行锁是通过给索引上的索引项加锁来实现的，而不是给表的行记录加锁实现的，这就意味着只有通过索引条件检索数据，InnoDB才使用行级锁，否则InnoDB将使用表锁。

2、由于InnoDB的行锁实现是针对索引字段添加的锁，不是针对行记录加的锁，因此虽然访问的是InnoDB引擎下表的不同行，但是如果使用相同的索引字段作为过滤条件，依然会发生锁冲突，只能串行进行，不能并发进行。

3、即使SQL中使用了索引，但是经过MySQL的优化器后，如果认为全表扫描比使用索引效率更高，此时会放弃使用索引，因此也不会使用行锁，而是使用表锁，比如对一些很小的表，MySQL就不会去使用索引。

间隙锁

当我们用范围条件而不是相等条件检索数据，并请求共享或排他锁时，InnoDB 会给符合条件的已有数据记录的索引项加锁；对于键值在条件范围内但并不存在的记录，叫做“间隙（GAP）”，InnoDB 也会对这个“间隙”加锁，这种锁机制就是所谓的间隙锁。举例来说，假如 user 表中只有 101 条记录，其 userid 的值分别是 1,2,...,100,101，下面的 SQL：

```
select * from user where userid > 100 for update;
```

是一个范围条件的检索，InnoDB 不仅会对符合条件的 userid 值为 101 的记录加锁，也会对userid 大于 101（但是这些记录并不存在）的“间隙”加锁，防止其它事务在表的末尾增加数据。

InnoDB使用间隙锁的目的，为了防止幻读，以满足串行化隔离级别的要求，对于上面的例子，要是不使用间隙锁，如果其他事务插入了 userid 大于 100 的任何记录，那么本事务如果再次执行上述语句，就会发生幻读。

意向共享锁和意向排他锁

意向共享锁（IS锁）：事务计划给记录加行共享锁，事务在给一行记录加共享锁前，必须先取得该表的IS 锁。

意向排他锁（IX锁）：事务计划给记录加行排他锁，事务在给一行记录加排他锁前，必须先取得该表的IX 锁。

	X	IX	S	IS
X	Conflict	Conflict	Conflict	Conflict
IX	Conflict	兼容	Conflict	兼容
S	Conflict	Conflict	兼容	兼容
IS	Conflict	兼容	兼容	兼容

1、意向锁是由InnoDB存储引擎获取行锁之前自己获取的

2、意向锁之间都是兼容的，不会产生冲突

3、意向锁存在的意义是为了更高效的获取表锁（表格中的X和S指的是表锁，不是行锁！！！）

4、意向锁是表级锁，协调表锁和行锁的共存关系。主要目的是显示事务正在锁定某行或者试图锁定某行。

InnoDB表级锁

在绝大部分情况下都应该使用行锁，因为事务和行锁往往是选择InnoDB的理由，但个别情况下也使用表级锁；

1) 事务需要更新大部分或全部数据，表又比较大，如果使用默认的行锁，不仅这个事务执行效率低，而且可能造成其他事务长时间等待和锁冲突；

2) 事务涉及多个表，比较复杂，很可能引起死锁，造成大量事务回滚。

如：

LOCK TABLE user READ; 读锁锁表

LOCK TABLE user WRITE; 写锁锁表

事务执行...

COMMIT/ROLLBACK; 事务提交或者回滚

UNLOCK TABLES; 本身自带提交事务，释放线程占用的所有表锁

死锁

MyISAM 表锁是 deadlock free 的，这是因为 MyISAM 总是一次获得所需的全部锁，要么全部满足，要么等待，因此不会出现死锁。但在 InnoDB 中，除单个 SQL 组成的事务外，锁是逐步获得的，即锁的粒度比较小，这就决定了在 InnoDB 中发生死锁是可能的。

```
mysql> select * from test_dead_lock where id=1 for update;
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

死锁问题一般都是我们自己的应用造成的，和多线程编程的死锁情况相似，大部分都是由于我们多个线程在获取多个锁资源的时候，获取的顺序不同而导致的死锁问题。因此我们应用在对数据库的多个表做更新的时候，不同的代码段，应对这些表按相同的顺序进行更新操作，以防止锁冲突导致死锁问题。

锁的优化建议

1. 尽量使用较低的隔离级别
2. 设计合理的索引并尽量使用索引访问数据，使加锁更加准确，减少锁冲突的机会提高并发能力
3. 选择合理的事务大小，小事务发生锁冲突的概率小
4. 不同的程序访问一组表时，应尽量约定以相同的顺序访问各表，对一个表而言，尽可能以固定的顺序存取表中的行。这样可以大大减少死锁的机会
5. 尽量用相等条件访问数据，这样可以避免间隙锁对并发插入的影响
6. 不要申请超过实际需要的锁级别
7. 除非必须，查询时不要显示加锁

MVCC多版本并发控制

MVCC是多版本并发控制（Multi-Version Concurrency Control，简称MVCC），是MySQL中基于乐观锁理论实现隔离级别的方式，用于实现已提交读和可重复读隔离级别的实现，也经常称为多版本数据库。MVCC机制会生成一个数据请求时间点的一致性数据快照（Snapshot），并用这个快照来提供一定级别（语句级或事务级）的一致性读取。从用户的角度来看，好像是数据库可以提供同一数据的多个版本（系统版本号和事务版本号）。

MVCC多版本并发控制中，读操作可以分为两类：

- 1、快照读（snapshot read）

读的是记录的可见版本，不用加锁。如select

2、当前读 (current read)

读取的是记录的最新版本，并且当前读返回的记录。如insert, delete, update, select...lock in share mode/for update

MVCC：每一行记录实际上有多个版本，每个版本的记录除了数据本身之外，增加了其它字段

DB_TRX_ID：记录当前事务ID

DB_ROLL_PTR：指向undo log日志上数据的指针

已提交读：每次执行语句的时候都重新生成一次快照 (Read View)，每次select查询时。

可重复读：同一个事务开始的时候生成一个当前事务全局性的快照 (Read View)，第一次select查询时。

快照内容读取原则：

- 1、版本未提交无法读取生成快照
- 2、版本已提交，但是在快照创建后提交的，无法读取
- 3、版本已提交，但是在快照创建前提交的，可以读取
- 4、当前事务内自己的更新，可以读到

MyISAM表级锁

MyISAM存储引擎不支持事务处理，因此它的并发比较简单，只支持到表锁的粒度，粒度比较大，并发能力一般，但不会引起死锁的问题，它支持表级共享的读锁和互斥的写锁。

表级锁

对 MyISAM 表的读操作（共享锁），不会阻塞其他用户对同一表的读请求，但会阻塞对同一表的写请求；对 MyISAM 表的写操作（排它锁），则会阻塞其他用户对同一表的读和写操作。

MyISAM 表的读操作与写操作之间，以及写操作之间是串行的。

MyISAM 在执行查询语句（SELECT）前，会自动给涉及的所有表加读锁，在执行更新操作（UPDATE、DELETE、INSERT 等）前，会自动给涉及的表加写锁，这个过程并不需要用户控制，是MySQL Server 端自动完成的。

并发插入优化

concurrent_insert优化。

```
mysql> show variables like 'concurrent_insert';
+-----+-----+
| variable_name | value |
+-----+-----+
| concurrent_insert | AUTO |
+-----+-----+
1 row in set (0.00 sec)
```

普通情况下，MyISAM的读操作和写操作都是串行的，但是其实MyISAM也是支持读和写的并发操作的，上面的concurrent_insert变量就是开关，允许一个线程在读的时候，另外一个线程在尾部进行插入（但是不能并发进行删除delete和更新update）。

锁调度优化

low_priority_updates优化。

在MyISAM存储引擎下，多个线程并发操作时，线程1试图获取读锁，线程2获取写锁，一般MyISAM认为写操作要比读操作重要，因此线程2几乎都会优先获取写锁，写操作完成后，线程1才会获取读锁。

即使线程1的读锁请求先到达，线程2的写锁请求后到达，那么线程2写锁的获取也会排在线程1读锁的前面。

因此，MyISAM存储引擎不适合大量的更新操作和查询操作，因为查询操作获取读锁的优先级比较低，会导致客户端查询获取结果的过程很慢。当然MySQL提供了很多参数设置，可以调整读锁的获取优先级，可以自行网上查阅深入了解。

MySQL优化

SQL和索引优化

应用优化

除了优化SQL和索引，很多时候，在实际生产环境中，由于数据库服务器本身的性能局限，就必须要对上层的应用来进行一些优化，使得上层应用访问数据库的压力能够减到最小。

连接池

应用上一般访问数据库，都是先和MySQL Server创建连接，然后发送SQL语句，Server处理完成后，再把结果通过网络返回给应用，然后关闭和MySQL Server的连接，因此短时间大量的数据库访问，消耗的TCP三次握手和四次挥手所花费的时间就很大了，稍微大一点的项目，我们都会在应用访问数据库的那一层上，添加连接池模块，相当于应用和MySQL Server事先创建一组连接，当应用需要请求MySQL Server时，不需要再进行TCP连接和释放连接了，一般连接池都会维护以下资源：

- 1、连接池里面保持固定数量的活跃TCP连接，供应用使用。
- 2、如果应用瞬间访问MySQL的量比较大，那么连接池会实时创建更多的连接给应用使用。
- 3、当连接池里面的TCP连接一段时间内没有被用到，连接池会释放多余的连接资源，保留它设置的最大空闲连接量就可以了。

连接池可以自己实现，也可以用第三方写好的库。

增加cache缓存层

业务上增加redis、memcache，一般用缓存把经常访问的数据缓存起来。

MySQL Server优化

对于MySQL Server端的优化，主要指的就是MySQL Server启动时加载的配置文件的配置项内容的优化（就是那个my.ini或者my.cnf），下面我们看看它的配置文件中有哪些是我们需要重点关注的优化参数。

MySQL查询缓存

MySQL的查询缓存是把select查询语句上一次的查询结果记录下来放在缓存当中，下一次再查询相同内容的时候，直接从缓存中取出来就可以了，不用再进行一遍真正的SQL查询。但是当两个select查询中间出现insert，update，delete语句的时候，查询缓存就会被清空。查询缓存适用更新不频繁的表，因为当表更新频繁的话，查询缓存也总是被清空，过多的查询缓存的数据添加和删除，就会影响MySQL的执行效率，还不如每次都从磁盘上查来得快（缓存指的就是一块内存，内存I/O比磁盘I/O快很多）。

可以在MySQL上通过以下命令，来查看查询缓存的设置：

```
mysql> show variables like '%query_cache%';
```

Variable_name	Value	
have_query_cache	YES	表示MySQL Server是否支持查询缓存，此处YES为支持！
query_cache_limit	1048576	
query_cache_min_res_unit	4096	
query_cache_size	1048576	表示查询缓存的大小，单位为M
query_cache_type	OFF	表示目前查询缓存目前未开启
query_cache_wlock_invalidate	OFF	

```
6 rows in set (0.00 sec)
```

通过show status命令，可以查看MySQL查询缓存的使用状况，如下：

```
mysql> show status like 'Qcache%';
```

Variable_name	Value	
Qcache_free_blocks	1	查询缓存中的空闲内存块的数目
Qcache_free_memory	1031832	查询缓存的空闲内存总数
Qcache_hits	0	
Qcache_inserts	0	被加入到缓存中的查询数目
Qcache_lowmem_prunes	0	因为缺少内存而被从缓存中删除的查询数目
Qcache_not_cached	170	
Qcache_queries_in_cache	0	在缓存中已注册的查询数目
Qcache_total_blocks	1	查询缓存中的块的总数目

```
8 rows in set (0.00 sec)
```

可以通过set命令设置上面的缓存参数开启MySQL查询缓存功能，也可以找到MySQL的配置文件（windows是my.ini，linux是my.cnf），修改query_cache_type参数为1就可以了，然后重启MySQL Server就可以使用了，如下：

```
query_cache_type=1
query_cache_size=10
```

修改完成，重启MySQL Server，查询缓存生效。

索引和数据缓存

主要指的就是innodb_buffer_pool_size配置项，从名字上就能看到，该配置项是针对InnoDB存储引擎起作用的，这个参数定义了InnoDB 存储引擎的表数据和索引数据的最大内存缓冲区大小。innodb_buffer_pool_size是同时为数据块和索引块做缓存，这个值设得越高，访问表中数据需要的磁盘 I/O 就越少。

```
innodb_buffer_pool_size=512M
```

MySQL线程缓存

主要指配置文件中thread_cache_size配置项。给大家讲过MySQL Server网络模块采用经典的I/O复用+线程池模型，之所以引入线程池，主要就是为了在业务执行的过程中，不会因为临时创建和销毁线程，造成系统性能降低，因为线程的创建和销毁是很耗费性能的，所以线程池就是在业务使用之前，先创建一组固定数量的线程，等待事件发生，当有SQL请求到达MySQL Server的时候，在线程池中取一个线程来执行该SQL请求就可以了，执行完成后，不销毁线程，而是把线程再归还到线程池中，等待下一次任务的处理（MySQL会根据连接量，自动加大线程池的数量）。

```
thread_cache_size=10
```

配置完thread_cache_size，重启MySQL Server服务生效。

并发连接数量和超时时间

MySQL Server作为一个服务器，可以设置客户端的最大连接量和连接超时时间，如果数据库连接统计数量比较大，这两个参数的值需要设置大一些。

```
mysql> show variables like '%connect%';
```

Variable_name	Value
character_set_connection	utf8
collation_connection	utf8_general_ci
connect_timeout	10
disconnect_on_expired_password	ON
init_connect	
max_connect_errors	100
max_connections	151
max_user_connections	0
performance_schema_session_connect_attrs_size	512

```
9 rows in set (0.01 sec)
```

在配置文件（my.cnf或my.ini）最下面，添加配置：max_connections=2000，然后重启MySQL Server，设置生效。

```
mysql> show global variables like '%timeout%';
```

Variable_name	Value
connect_timeout	10
delayed_insert_timeout	300
have_statement_timeout	YES
innodb_flush_log_at_timeout	1
innodb_lock_wait_timeout	50
innodb_rollback_on_timeout	OFF
interactive_timeout	28800
lock_wait_timeout	31536000
net_read_timeout	30
net_write_timeout	60
rpl_stop_slave_timeout	31536000
slave_net_timeout	60
wait_timeout	28800

```
13 rows in set (0.00 sec)
```

MySQL Server对于超时未通信的连接，进行主动关闭操作。设置超时时间，超过设置时间没有请求就主动断开，单位是秒，在配置文件中添加配置：wait_timeout = 600。

MySQL日志

```
mysql> show variables like 'log_%'; # 全局变量 设置&状态 show variables & show
status
+-----+-----+
--+
| variable_name          | Value
|
+-----+-----+
--+
| log_bin                | OFF
|
| log_bin_basename       |
|
| log_bin_index          |
|
| log_bin_trust_function_creators | OFF
|
| log_bin_use_v1_row_events | OFF
|
| log_builtin_as_identified_by_password | OFF
|
| log_error               |
/usr/local/mysql/data/mysqld.local.err |
| log_error_verbosity    | 3
|
| log_output              | FILE
|
| log_queries_not_using_indexes | OFF
|
| log_slave_updates       | OFF
|
| log_slow_admin_statements | OFF
|
| log_slow_slave_statements | OFF
|
| log_statements_unsafe_for_binlog | ON
|
| log_syslog              | OFF
|
| log_syslog_facility     | daemon
|
| log_syslog_include_pid  | ON
|
| log_syslog_tag          |
|
| log_throttle_queries_not_using_indexes | 0
|
| log_timestamps          | UTC
|
| log_warnings            | 2
|
```



```
+-----+
--+
21 rows in set (0.00 sec)
```

打开my.ini,在后面加上上面的参数，保存后重启mysql服务就行了。

```
#Enter a name for the error log file. Otherwise a default name will be used.
log-error=err.log
#Enter a name for the query log file. Otherwise a default name will be used.
#log=
#Enter a name for the slow query log file. Otherwise a default name will be
used.
#log-slow-queries=
#Enter a name for the update log file. Otherwise a default name will be used.
#log-update=
#Enter a name for the binary log. Otherwise a default name will be used.
#log-bin=
```

在linux root下重启mysqld服务：service mysqld restart

错误日志

错误日志是 MySQL 中最重要的日志之一，它记录了当 mysqld 启动和停止时，以及服务器在运行过程中发生任何严重错误时的相关信息。当数据库出现任何故障导致无法正常使用时，可以首先查看此日志。

mysqld 使用错误日志名 host_name.err(host_name 为主机名) 并默认在参数 DATADIR(数据目录)指定的目录中写入日志文件。

查询日志

查询日志记录了客户端的所有语句。由于上线项目sql特别多，开启查询日志IO太多导致MySQL效率低，只有在调试时才开启，比如通过查看sql发现热点数据进行缓存。

```
mysql> show global variables like "%genera%";
```

二进制日志

二进制日志(BINLOG)记录了所有的 DDL(数据定义语言)语句和 DML(数据操纵语言) 语句，但是不包括数据查询语句。语句以“事件”的形式保存，它描述了数据的更改过程。此日志对于灾难时的数据恢复起着极其重要的作用。

两个重要的应用场景：主从复制、数据恢复

查看binlog：mysql> show binary logs;

通过mysqlbinlog工具（mysql原生自带的工具）可以快速解析大量的binlog日志文件，如：

```
shell> mysqlbinlog --no-defaults --database=school --base64-output=decode-rows
-v --start-datetime='2021-05-01 00:00:00' --stop-datetime='2021-05-10 00:00:00'
mysql-bin.000001 | more
```

慢查询日志

MySQL可以设置慢查询日志，当SQL执行的时间超过我们设定的时间，那么这些SQL就会被记录在慢查询日志当中，然后通过查看日志，用explain分析这些SQL的执行计划，来判定为什么效率低下，是没有使用到索引？还是索引本身创建的有问题？或者是索引使用到了，但是由于表的数据量太大，花费的时间就是很长，那么此时我们可以把表分成n个小表，比如订单表按年份分成多个小表等。

慢查询日志相关的参数如下所示：

```
mysql> show variables like '%slow_query%';
+-----+-----+
| variable_name | value |
+-----+-----+
| slow_query_log | OFF |
| slow_query_log_file | /usr/local/mysql/data/LeideMacBook-Pro-slow.log |
+-----+-----+
2 rows in set (0.00 sec)
```

慢查询日志记录了包含所有执行时间超过参数 long_query_time（单位：秒）所设置值的 SQL语句的日志，在MySQL上用命令可以查看，如下：

```
mysql> show variables like 'long%';
+-----+-----+
| variable_name | value |
+-----+-----+
| long_query_time | 10.000000 |
+-----+-----+
1 row in set (0.00 sec)
```

这个值是可以修改的，如下：

```
mysql> set long_query_time = 1; #注意，单位是秒
Query OK, 0 rows affected (0.00 sec)
```

现在修改成超过1秒的SQL都会被记录在慢查询日志当中！可以设置为0.01秒，表示10毫秒。

慢查询日志，默认名称是host_name-slow.log，存放在MySQL的数据路径下，内容格式显示大致如下：

```
Query_time: 0.012000 Lock_time: 0.000000 Rows_sent: 0 Rows_examined: 139

use tuluneducation;
SET timestamp=1534527397;
select id,author from subject where content like '%linux%' and title like '%c++
linux%';
```

通过查询慢查询日志，发现项目运行过程中，上面这条SQL语句的执行时间超过了设定的慢查询时间，那么接下来就需要用explain分析一下该SQL的执行计划了，根据具体情况找出SQL和索引该怎么去优化。

show profiles命令可有查看sql具体的运行时间，全局变量的名字是：profiling

redo log和undo log

redo log

redo log: 重做日志, 用于记录事务操作的变化, 确保事务的持久性。redo log是在事务开始后就开始记录, 不管事务是否提交都会记录下来, 在异常发生时(如数据持久化过程中掉电), InnoDB会使用redo log恢复到掉电前的时刻, 保证数据的完整性。

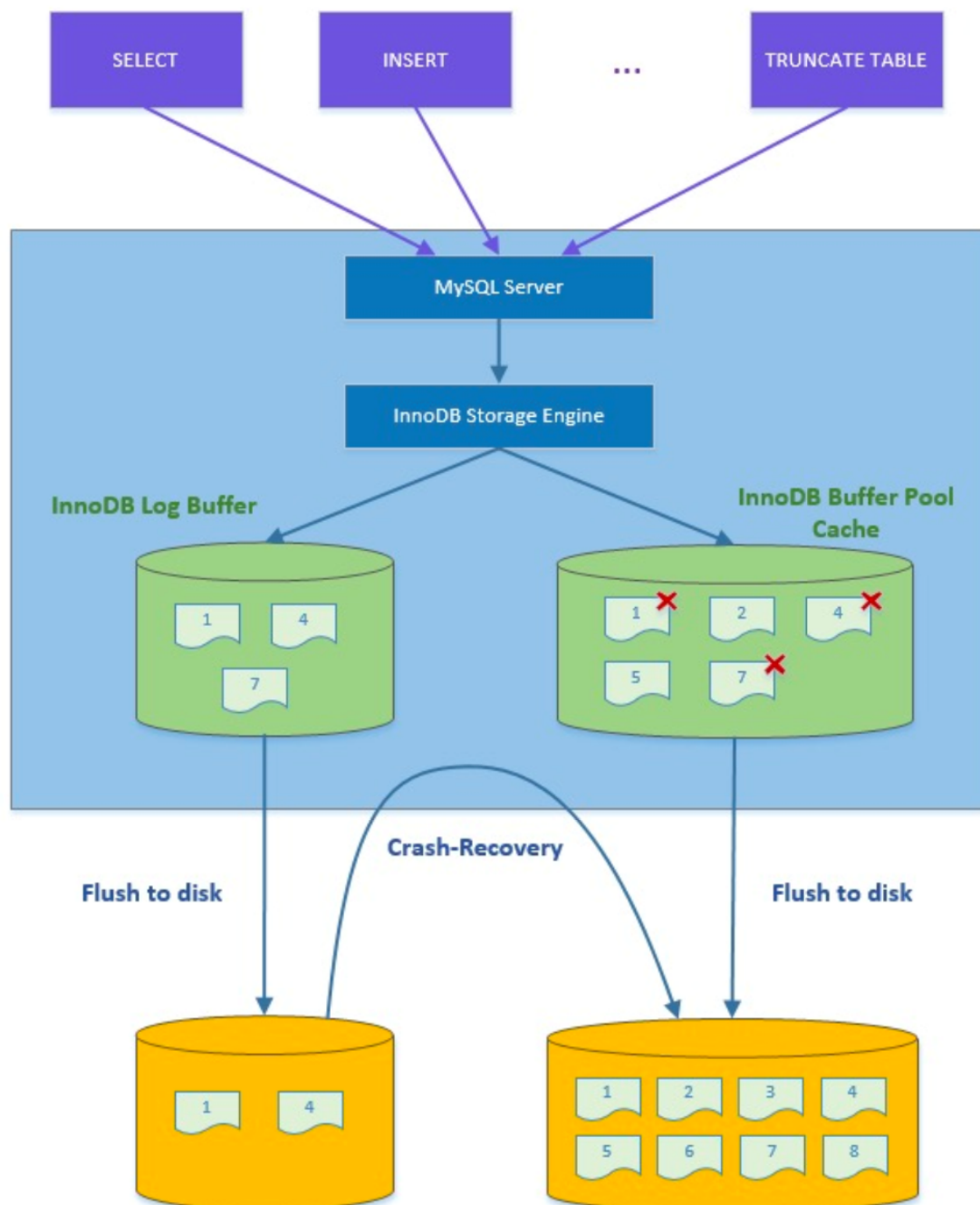
innodb_log_buffer_size默认是16M, 就是redo log缓冲区的大小, 它随着事务开始, 就开始写redo log, 如果事务比较大, 为了避免事务执行过程中花费过多磁盘IO, 可以设置比较大的redo log缓存, 节省磁盘IO。

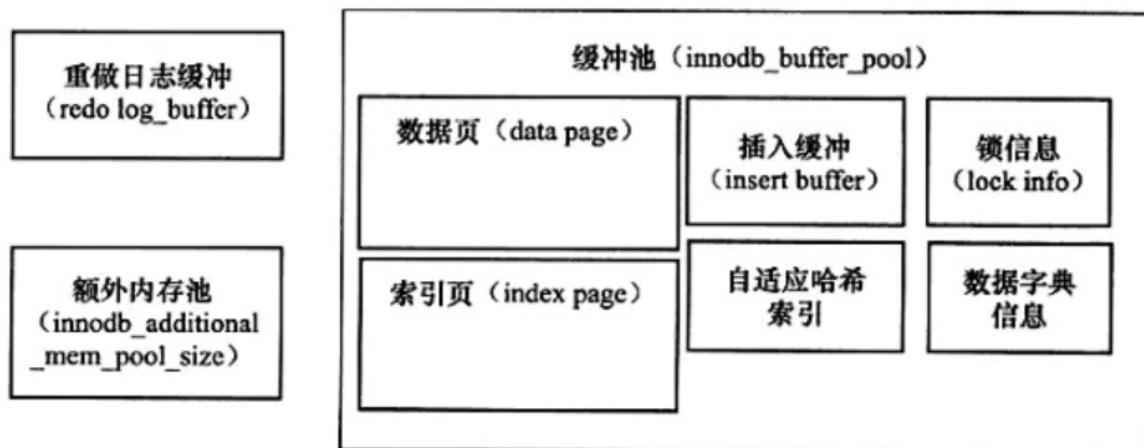
InnoDB修改操作数据, 不是直接修改磁盘上的数据, 实际只是修改Buffer Pool中的数据。InnoDB总是先把Buffer Pool中的数据改变记录到redo log中, 用来进行崩溃后的数据恢复。优先记录redo log, 然后再慢慢的将Buffer Pool中的脏数据刷新到磁盘上。

innodb_log_group_home_dir指定的目录下的两个文件: ib_logfile0, ib_logfile1, 该文件被称作重做日志。

buffer pool缓存池:

作用: 加速读和加速写, 直接操作data page, 写redo log修改就算完成, 有专门的线程去做把buffer pool中的dirty page写入磁盘。

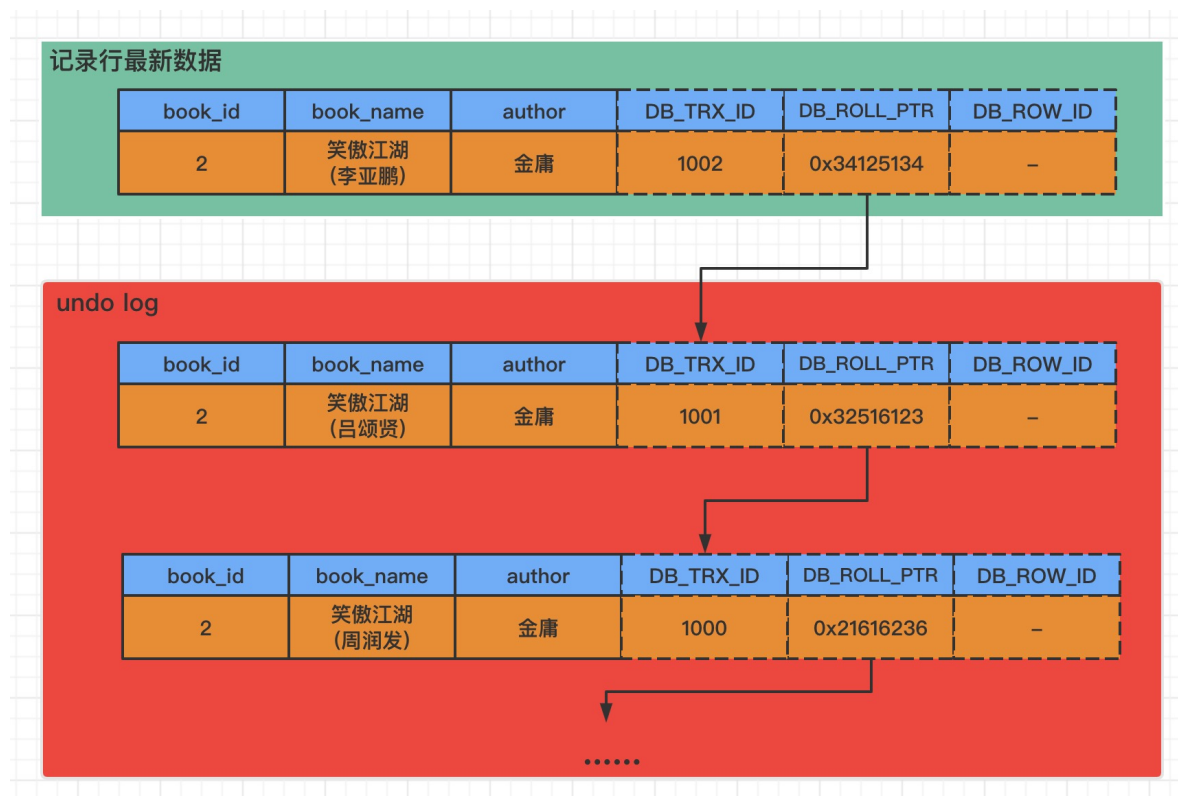




undo log

undo log: 回滚日志, 保存了事务发生之前的数据的一个版本, 用于事务执行时的回滚操作, 同时也是实现多版本并发控制 (MVCC) 下读操作的关键技术。

DB_TRX_ID:事务ID DB_ROLL_PTR:回滚指针



数据备份和GUI工具

数据备份恢复常用命令

```
mysqldump -u root -p123456 --all-databases > ~/all.sql
```

```
mysqldump -u root -p123456 --databases school > ~/school.sql
```

```
mysqldump -u root -p123456 school user > ~/user.sql
```

```
mysql -u root -p123456 -D school -e 'select * from user where age>18' > ~/user_data.txt
```

```
source ~/school.sql
```

GUI工具

SQLyog

Navicat

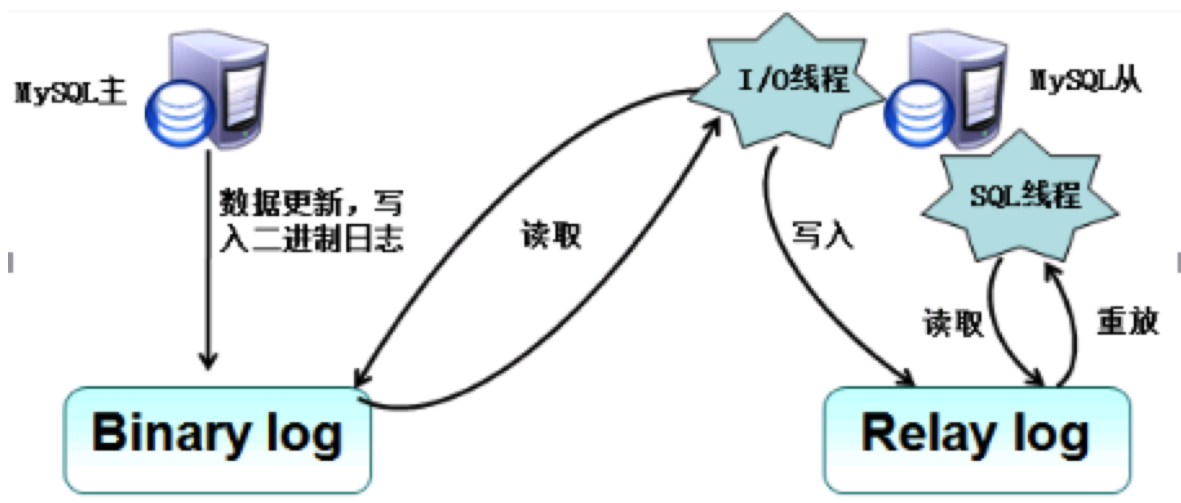
MySQL集群

在实际生产环境中，如果对mysql数据库的读和写都在一台数据库服务器中操作，无论是在安全性、高可用性，还是高并发等各个方面都是不能满足实际需求的，一般要通过主从复制的方式来同步数据，再通过读写分离来提升数据库的并发负载能力。

- 1、数据备份 - 热备份&容灾&高可用
- 2、读写分离，支持更大的并发

主从复制

原理介绍



主从复制的流程：两个日志（binlog二进制日志&relay log日志）和三个线程（master的一个线程和slave的二个线程）

- 1、主库的更新操作写入binlog二进制日志中。
- 2、master服务器创建一个binlog转储线程，将二进制日志内容发送到从服务器。
- 3、slave机器执行START SLAVE命令会在从服务器创建一个IO线程，接收master的binary log复制到其中继日志。
首先slave开始一个工作线程（I/O线程），I/O线程在master上打开一个普通的连接，然后开始binlog dump process，binlog dump process从master的二进制日志中读取事件，如果已经跟上master，它会睡眠并等待master产生新的事件，I/O线程将这些事件写入中继日志。
- 4、sql slave thread（sql从线程）处理该过程的最后一步，sql线程从中继日志中读取事件，并重放其中的事件而更新slave机器的数据，使其与master的数据一致。只要该线程与I/O线程保持一致，中继日志通常会位于os缓存中，所以中继日志的开销很小。

配置命令

条件：master和slave机器的信息

master (centos7) : 192.168.131.129

slave (win10) : 192.168.0.6

保证master和slave之间的网络互通，并且保证3306端口是开放的。

master配置：

1、开启二进制日志

配置log_bin和全局唯一的server-id。

2、创建一个用于主从库通信的账号

```
mysql> CREATE USER 'mslave'@'192.168.131.1' IDENTIFIED BY '1qaz@WSX';
mysql> GRANT REPLICATION SLAVE ON *.* to 'mslave'@'192.168.131.1' IDENTIFIED BY '1qaz@WSX';
mysql> FLUSH PRIVILEGES;
```

3、获取binlog的日志文件名和position

```
mysql> show master status;
```

slave配置：

1、配置全局唯一的server-id（涉及修改配置文件，需要重启mysql57服务）

2、使用master创建的账户读取binlog同步数据（stop slave; start slave）

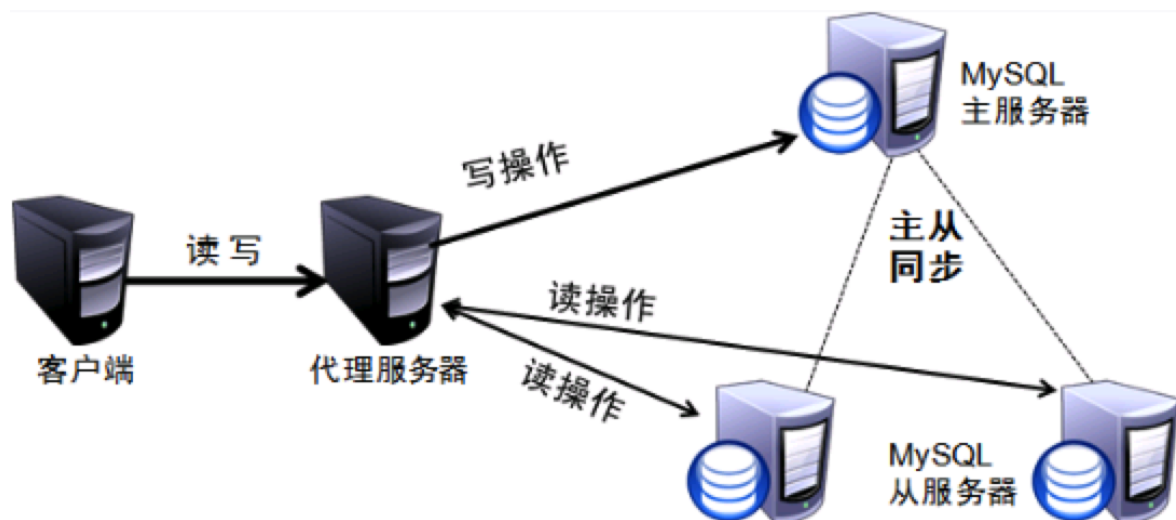
```
mysql> CHANGE MASTER TO MASTER_HOST='192.168.131.129',
MASTER_PORT=3306,
MASTER_USER='mslave',
MASTER_PASSWORD='1qaz@WSX',
MASTER_LOG_FILE='mysql-bin.000006',
MASTER_LOG_POS=1106;
```

3、START SLAVE

通过show slave status命令查看主从复制状态。show processlist查看master和slave相关线程的运行状态。

读写分离

读写分离就是在主服务器上修改，数据会同步到从服务器，从服务器只能提供读取数据，不能写入，实现备份的同时也实现了数据库性能的优化，以及提升了服务器安全。



目前较为常见的MySQL读写分离方式有：

- 程序代码内部实现
- 引入中间代理层
 - MySQL_proxy
 - Mycat

Mycat读写分离配置

条件：

master (centos7) : 192.168.131.129

slave (win10) : 192.168.31.27

1、JDK1.7版本以上 (java -version检查jdk环境)

2、MySQL的root账户有远程访问权限

安装与配置：

1、安装Mycat

- Mycat-server-1.6-RELEASE-20161028204710-linux.tar.gz (见课程附件资料 rz工具上传到linux, tar -zxvf解压)
- 解压放到/usr/local/mycat目录下
- ./mycat start启动服务，默认在8066和9066端口

2、配置文件

server.xml：配置登录Mycat的账号信息

```
<user name="root">
  <property name="password">123456</property>
  <property name="schemas">USERDB</property>
</user>
```

schema.xml：配置逻辑库和数据源、读写分离、分库分表信息等

```
<?xml version="1.0"?>
<!DOCTYPE mycat:schema SYSTEM "schema.dtd">
<mycat:schema xmlns:mycat="http://io.mycat/">
  <!-- 逻辑数据库 -->
  <schema name="USERDB" checkSQLschema="false" sqlMaxLimit="100"
dataNode="dn1"></schema>
  <!-- 存储节点 -->
  <dataNode name="dn1" dataHost="node1" database="mytest" />
  <!-- 数据库主机 -->
  <dataHost name="node1" maxCon="1000" minCon="10" balance="3"
writeType="0" dbType="mysql" dbDriver="native"
switchType="1" slaveThreshold="100">
    <heartbeat>select user()</heartbeat>
    <!-- can have multi write hosts -->
    <writeHost host="192.168.131.129" url="192.168.131.129:3306"
user="root"
password="123456">
        <!-- can have multi read hosts -->
        <readHost host="192.168.31.27" url="192.168.31.27:3306"
user="root" password="123456" />
    </writeHost>
```



```
<!-- 写数据库宕机后的备份数据库 -->
<writeHost host="192.168.31.27" url="192.168.31.27:3306"
user="root"
password="123456" />
</dataHost>
</mycat:schema>
```

balance:

"0": 不开启读写分离

"1": 全部的readHost和stand by writeHost参与select语句的负载

"2": 所有读操作随机在readHost和writeHost上分发

"3": 所有读请求随机分发到writeHost对应的readHost上执行

writeType="0": 所有写操作发送到配置的第一个writeHost, 第一个挂掉切换到还生存的第二个writeHost

switchType:

"-1": 不自动切换

"1": 自动切换, 根据心跳select user()

"2": 基于MySQL的主从同步状态决定是否进行切换 show slave status

3、端口

8066: 数据端口

9066: 管理端口

【扩展】MySQL分库分表

数据库架构演变

刚开始多数项目用单机数据库就够了, 随着服务器流量越来越大, 面对的请求也越来越多, 我们做了数据库读写分离, 使用多个从库副本 (Slave) 负责读, 使用主库 (Master) 负责写, master和slave通过主从复制实现数据同步更新, 保持数据一致。slave 从库可以水平扩展, 所以更多的读请求不成问题。

但是当用户量级上升, 写请求越来越多, 怎么保证数据库的负载足够? 增加一个Master是不能解决问题的, 因为数据要保存一致性, 写操作需要2个master之间同步, 相当于是重复了, 而且架构设计更加复杂。

这时需要用到分库分表 (sharding), 对写操作进行切分。

库表问题

单库太大

单库处理能力有限、所在服务器上的磁盘空间不足、遇到IO瓶颈, 需要把单库切分成更多更小的库

单表太大

CURD效率都很低、数据量太大导致索引膨胀、查询超时, 需要把单表切分成多个数据集更小的表

拆分策略

单个库太大, 先考虑是表多还是数据多:

- 如果因为表多而造成数据过多，则使用垂直拆分，即根据业务拆分成不同的库
- 如果因为单张表的数据量太大，则使用水平拆分，即把表的数据按照某种规则拆分成多张表

分库分表的原则应该是先考虑垂直拆分，再考虑水平拆分。

垂直拆分

server.xml

```
<user name="root">
  <property name="password">123456</property>
  <property name="schemas">USERDB1,USERDB2</property>
</user>
```

schema.xml

```
<?xml version="1.0"?>
<!DOCTYPE mycat:schema SYSTEM "schema.dtd">
<mycat:schema xmlns:mycat="http://io.mycat/">
  <!-- 逻辑数据库 -->
  <schema name="USERDB1" checkSQLschema="false" sqlMaxLimit="100"
dataNode="dn1" />
  <schema name="USERDB2" checkSQLschema="false" sqlMaxLimit="100"
dataNode="dn2" />
  <!-- 存储节点 -->
  <dataNode name="dn1" dataHost="node1" database="mytest1" />
  <dataNode name="dn2" dataHost="node2" database="mytest2" />
  <!-- 数据库主机 -->
  <dataHost name="node1" maxCon="1000" minCon="10" balance="0"
writeType="0" dbType="mysql" dbDriver="native">
    <heartbeat>select user()</heartbeat>
    <writeHost host="192.168.131.129" url="192.168.131.129:3306"
user="root"
password="123456">
    </writeHost>
  </dataHost>
  <dataHost name="node2" maxCon="1000" minCon="10" balance="0"
writeType="0" dbType="mysql" dbDriver="native">
    <heartbeat>select user()</heartbeat>
    <writeHost host="192.168.0.6" url="192.168.0.6:3306" user="root"
password="123456" />
  </dataHost>
</mycat:schema>
```

垂直分表

也就是“大表拆小表”，基于列字段进行。

一般是表中的字段较多，将不常用的，数据较大，长度较长（比如text类型字段）的拆分到“扩展表”。

一般是针对几百列的这种大表，也避免查询时，数据量太大造成的“跨页”问题。

垂直分库

垂直分库针对的是一个系统中的不同业务进行拆分。

比如用户User一个库，商品Product一个库，订单Order一个库，切分后，要放在多个服务器上，而不是一个服务器上。想象一下，一个购物网站对外提供服务，会有用户，商品，订单等的CRUD。没拆分之前，全部都是落到单一的库上的，这会让数据库的单库处理能力成为瓶颈。按垂直分库后，如果还是

放在一个数据库服务器上，随着用户量增大，这会让单个数据库的处理能力成为瓶颈，还有单个服务器的磁盘空间，内存，tps等非常吃紧。所以我们要拆分到多个服务器上，这样上面的问题都解决了，以后也不会面对单机资源问题。

数据库业务层面的拆分，和服务的“治理”，“降级”机制类似，也能对不同业务的数据分别的进行管理，维护，监控，扩展等。数据库往往最容易成为应用系统的瓶颈，而数据库本身属于“有状态”的，相对于Web和应用服务器来讲，是比较难实现“横向扩展”的。数据库的连接资源比较宝贵且单机处理能力也有限，在高并发场景下，垂直分库一定程度上能够突破IO、连接数及单机硬件资源的瓶颈。

水平拆分

server.xml

```
<user name="root">
  <property name="password">123456</property>
  <property name="schemas">USERDB</property>
</user>
```

schema.xml

```
<?xml version="1.0"?>
<!DOCTYPE mycat:schema SYSTEM "schema.dtd">
<mycat:schema xmlns:mycat="http://io.mycat/">
  <!-- 逻辑数据库 -->
  <schema name="USERDB" checkSQLschema="false" sqlMaxLimit="100">
    <table name="user" dataNode="dn1" />
    <table name="student" primaryKey="id" autoIncrement="true"
dataNode="dn1,dn2" rule="mod-long"/>
  </schema>
  <!-- 存储节点 -->
  <dataNode name="dn1" dataHost="node1" database="mytest1" />
  <dataNode name="dn2" dataHost="node2" database="mytest2" />
  <!-- 数据库主机 -->
  <dataHost name="node1" maxCon="1000" minCon="10" balance="0"
writeType="0" dbType="mysql" dbDriver="native">
    <heartbeat>select user()</heartbeat>
    <writeHost host="192.168.131.129" url="192.168.131.129:3306"
user="root"
password="123456">
    </writeHost>
  </dataHost>
  <dataHost name="node2" maxCon="1000" minCon="10" balance="0"
writeType="0" dbType="mysql" dbDriver="native">
    <heartbeat>select user()</heartbeat>
    <writeHost host="192.168.0.6" url="192.168.0.6:3306" user="root"
password="123456" />
  </dataHost>
</mycat:schema>
```

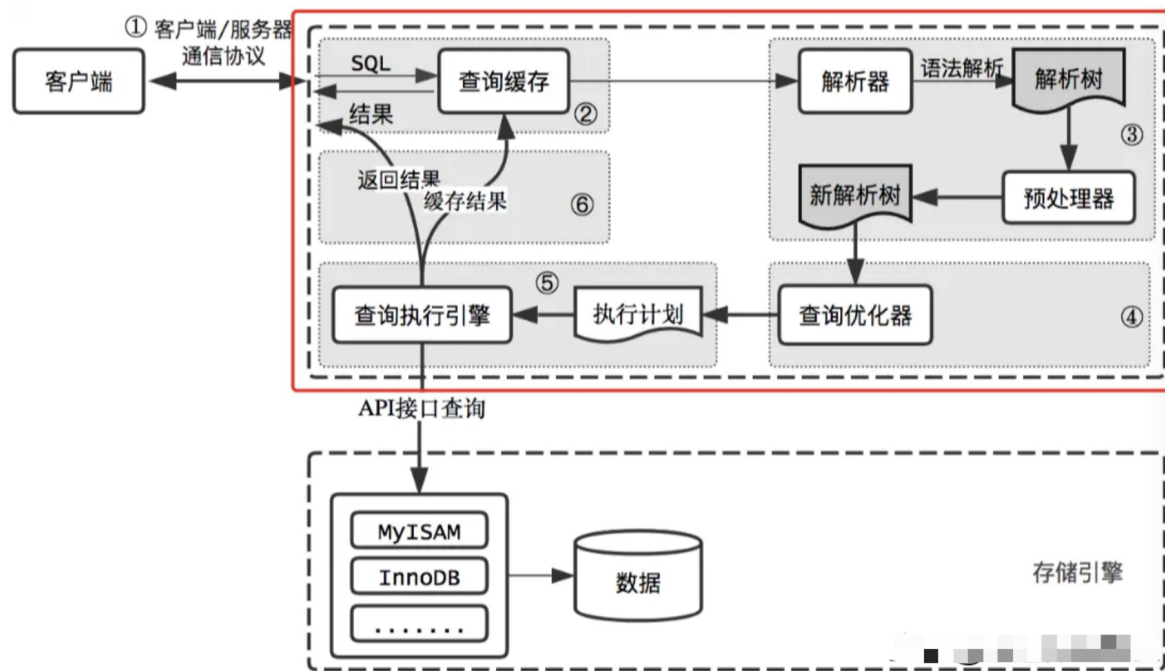
水平分表

针对数据量巨大的单张表（比如订单表），按照某种规则（RANGE,HASH取模等），切分到多张表里面去。但是这些表还是在同一个库中，所以库级别的数据库操作还是有IO瓶颈，不建议采用。

水平分库分表

将单张表的数据切分到多个服务器上去，每个服务器具有相应的库与表，只是表中数据集合不同。水平分库分表能够有效的缓解单机和单库的性能瓶颈和压力，突破IO、连接数、硬件资源等的瓶颈。

【扩展】SQL详细处理流程



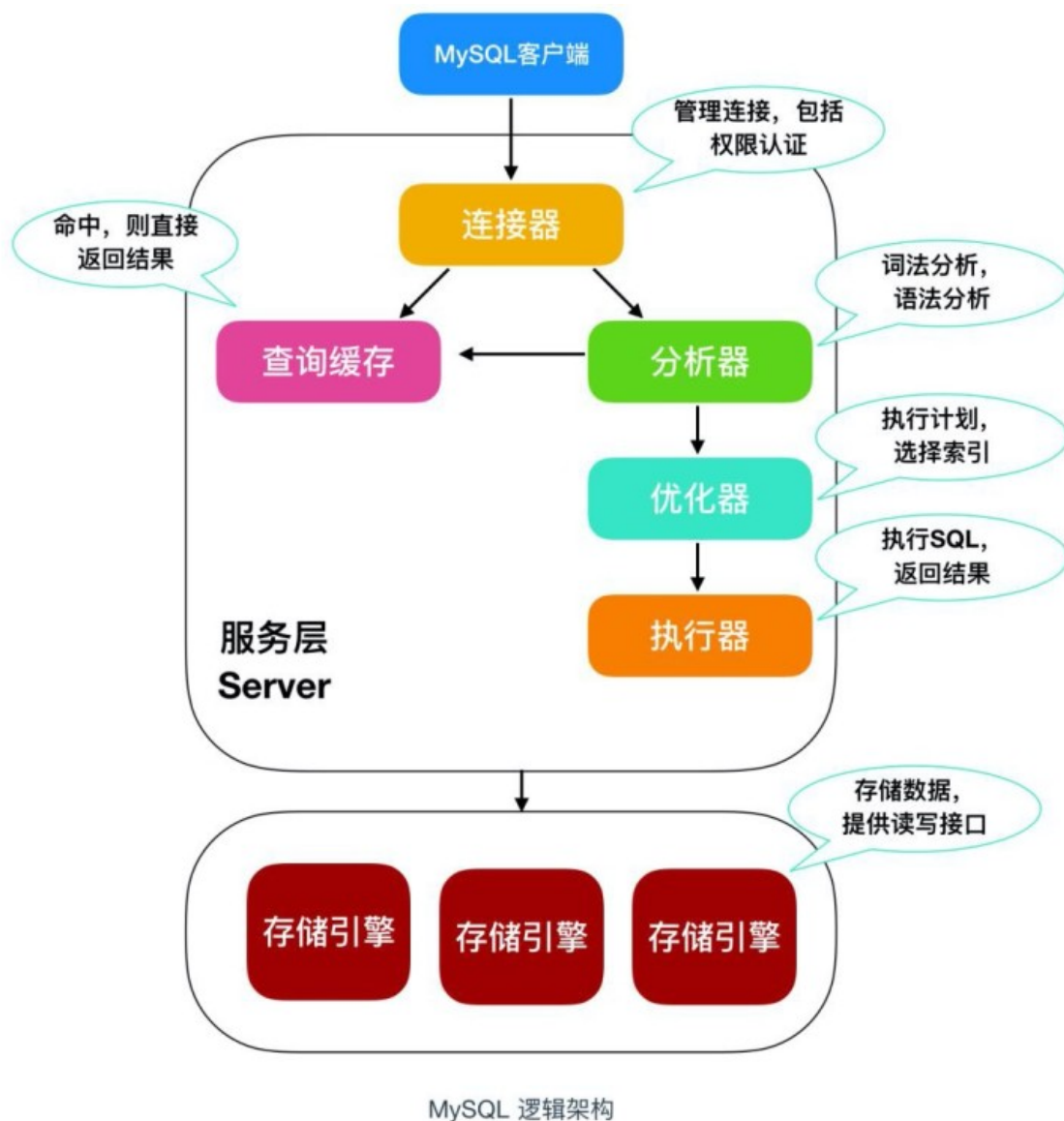
连接器: 管理连接，权限验证

解析器: 词法以及语法分析

优化器: 生成执行计划，选择合适索引

执行器: 操作引擎获取结果

存储引擎: 存储数据，提供读写接口



【扩展】MySQL常用操作

设置字符编码

有时候MySQL中的表需要存储中文，需要设置表的字符编码为utf8，否则默认的字符编码有可能不能正确处理中文，在MySQL中设置表的字符编码如下：

通过命令查看MySQL表的字符编码，如下：

```
mysql> show variables like 'charac%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| character_set_client | utf8 |
| character_set_connection | utf8 |
| character_set_database | utf8 |
```

```

| character_set_filesystem | binary
|
| character_set_results    | utf8
|
| character_set_server     | utf8
|
| character_set_system     | utf8
|
| character_sets_dir       | /usr/local/mysql-5.7.31-macos10.14-
x86_64/share/charsets/ |
+-----+-----+-----+-----+-----+-----+
-----+
8 rows in set (0.00 sec)

```

如下设置:

```

mysql> set character_set_server=utf8;
Query OK, 0 rows affected (0.00 sec)

```

字符编码设置utf8成功, MySQL的status运行状态和运行参数都是通过全局变量来控制的, 用show status和show variables两个命令可以查看这些信息, 用set可以设置这些信息, 改变配置。

设置存储引擎

如何查看当前MySQL版本都支持哪些存储引擎呢? 如下命令:

```

mysql> show engines\G;
***** 1. row *****
      Engine: InnoDB
      Support: DEFAULT
      Comment: Supports transactions, row-level locking, and foreign keys
Transactions: YES
          XA: YES
      Savepoints: YES
***** 2. row *****
      Engine: MRG_MYISAM
      Support: YES
      Comment: Collection of identical MyISAM tables
Transactions: NO
          XA: NO
      Savepoints: NO
***** 3. row *****
      Engine: PERFORMANCE_SCHEMA
      Support: YES
      Comment: Performance Schema
Transactions: NO
          XA: NO
      Savepoints: NO
***** 4. row *****
      Engine: BLACKHOLE
      Support: YES
      Comment: /dev/null storage engine (anything you write to it disappears)
Transactions: NO
          XA: NO
      Savepoints: NO
***** 5. row *****

```

```

    Engine: CSV
    Support: YES
    Comment: CSV storage engine
Transactions: NO
    XA: NO
    Savepoints: NO
***** 6. row *****
    Engine: MyISAM
    Support: YES
    Comment: MyISAM storage engine
Transactions: NO
    XA: NO
    Savepoints: NO
***** 7. row *****
    Engine: ARCHIVE
    Support: YES
    Comment: Archive storage engine
Transactions: NO
    XA: NO
    Savepoints: NO
***** 8. row *****
    Engine: MEMORY
    Support: YES
    Comment: Hash based, stored in memory, useful for temporary tables
Transactions: NO
    XA: NO
    Savepoints: NO
***** 9. row *****
    Engine: FEDERATED
    Support: NO
    Comment: Federated MySQL storage engine
Transactions: NULL
    XA: NULL
    Savepoints: NULL
9 rows in set (0.01 sec)

```

Support后面是DEFAULT，表示当前使用的存储引擎，可以看到上面是InnoDB存储引擎。

可以在创建表的时候指定存储引擎，如下：

```

CREATE TABLE ai (
  i BIGINT(20) NOT NULL AUTO_INCREMENT,
  PRIMARY KEY (i)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

```

也可以修改已存在表的存储引擎，如下：

```

ALTER TABLE user ENGINE = InnoDB;

```

还可以修改配置文件，MySQL Server启动的时候都会加载一个配置文件，windows下是my.ini文件，linux下是my.cnf文件，打开配置文件，在[mysqld]下面添加如下内容，保存，重启MySQL Server服务器，默认就采用配置项指定的存储引擎了。

```

default-storage-engine=INNODB

```

数据备份与恢复

通过mysqldump命令把指定db里面的表进行备份，写入sql脚本中：

```
mysqldump -h 9.84.205.186 -P 3306 -u root -p123456 db_name table_name > data_log.sql
```

登录mysql后，通过source命令可以把sql脚本的数据恢复到db中：

```
source /home/localuser/data_log.sql
```

还可以直接备份sql查询的数据：

```
mysql -h 9.84.205.186 -P 3306 -u root -p123456 -D db_name -e "select * from db_table" > data_log.txt
```

设置支持ip远程连接

mysql默认只能通过localhost访问，不能通过ip远程访问，主要考虑安全的问题。如果在集群分布式环境中单独部署mysql，就需要设置通过ip远程访问了

```
mysql> select Host,User,authentication_string from user;
+-----+-----+-----+
| Host      | User      | authentication_string |
+-----+-----+-----+
| localhost | root      | *6BB4837EB74329105EE4568DDA7DC67ED2CA2AD9 |
| localhost | mysql.session | *THISISNOTAVALIDPASSWORDTHATCANBEUSEDHERE |
| localhost | mysql.sys   | *THISISNOTAVALIDPASSWORDTHATCANBEUSEDHERE |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

上面显示可以看到，mysql默认只支持localhost访问。

```
mysql> grant all privileges on *.* to 'root'@%' identified by '123456' with
grant option;
Query OK, 0 rows affected, 1 warning (0.00 sec)
mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)
mysql> quit
Bye
[root@localhost Downloads]# service mysqld restart
Redirecting to /bin/systemctl restart mysqld.service
```

通过ip地址远程连接mysql如下：

```
mysql -h 172.20.10.3 -u root -p123456
```

创建账户

创建新账户和密码，只能在本地访问：


```
mysql> create user 'ms'@'localhost' identified by '123456';
```

创建新账户和密码，可以通过任意或者指定IP访问：

```
mysql> create user 'ms'@'%' identified by '123456';
```

新用户创建完成后，刷新库表赋予权限：

```
mysql> grant all privileges on *.* to 'tony'@'%' identified by '123456' with  
grant option;  
mysql> flush privileges;
```

开启centos防火墙3306端口

centos7防火墙配置不再是iptables了，配置如下：

```
# 开放3306端口  
firewall-cmd --zone=public --add-port=3306/tcp --permanent  
# 重启防火墙  
firewall-cmd --reload  
# 查看当前开放的端口列表  
firewall-cmd --list-ports
```

可以在win下测试是否通过mysql命令能够成功连接centos的mysqld服务。

```
systemctl start firewalld.service
```

```
systemctl stop firewalld.service
```

```
systemctl enable firewalld.service
```

```
systemctl disable firewalld.service
```

```
systemctl status firewalld.service
```

MySQL面试问题

- 范式设计
- SQL
- 索引
- 存储引擎
- 事务
- 锁+MVCC
- 日志
- 主从复制+读写分离
- 数据库中间件
- 分库分表

可以重点关注面经上的数据库问题！